

Redes Neurais Informadas por Física em Mecânica Computacional

Tópicos Especiais em Mecânica dos Sólidos

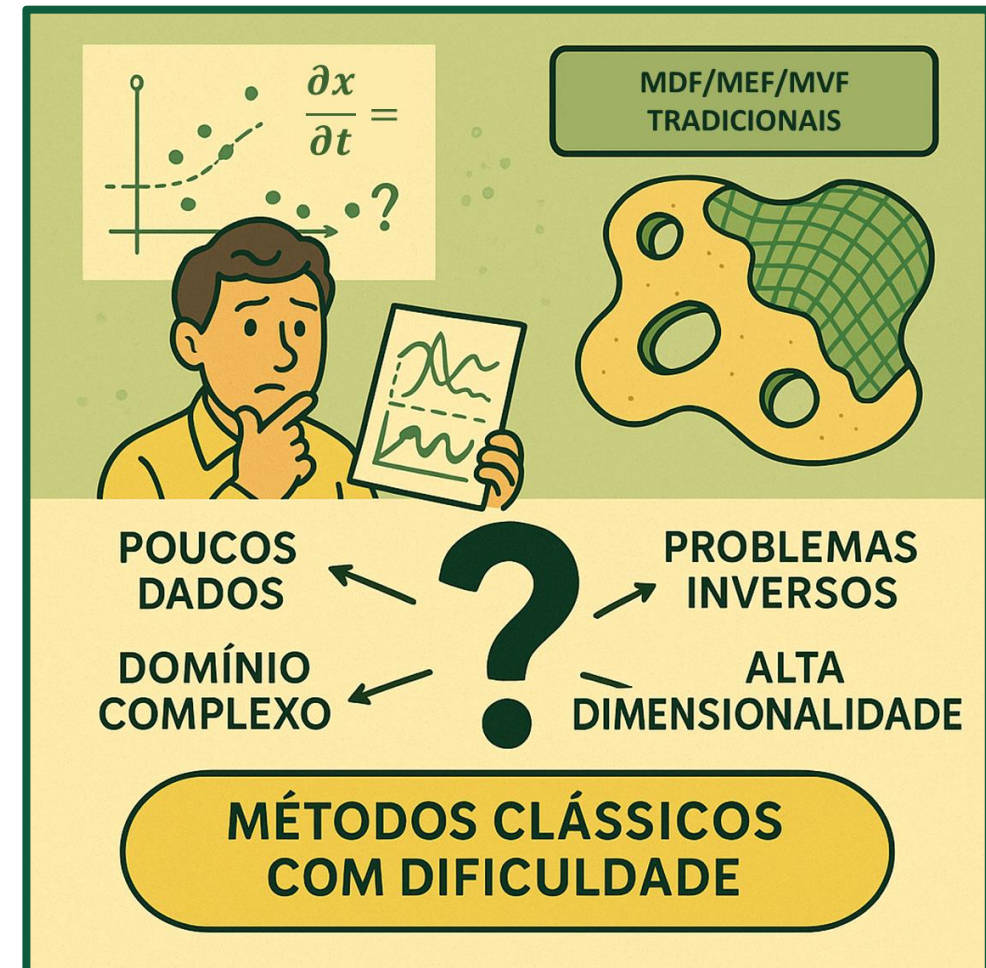
Eng. Paulo de Souza Silva e Prof. Dr. Wallace Gusmão

Junho de 2025



INTRODUÇÃO E MOTIVAÇÃO

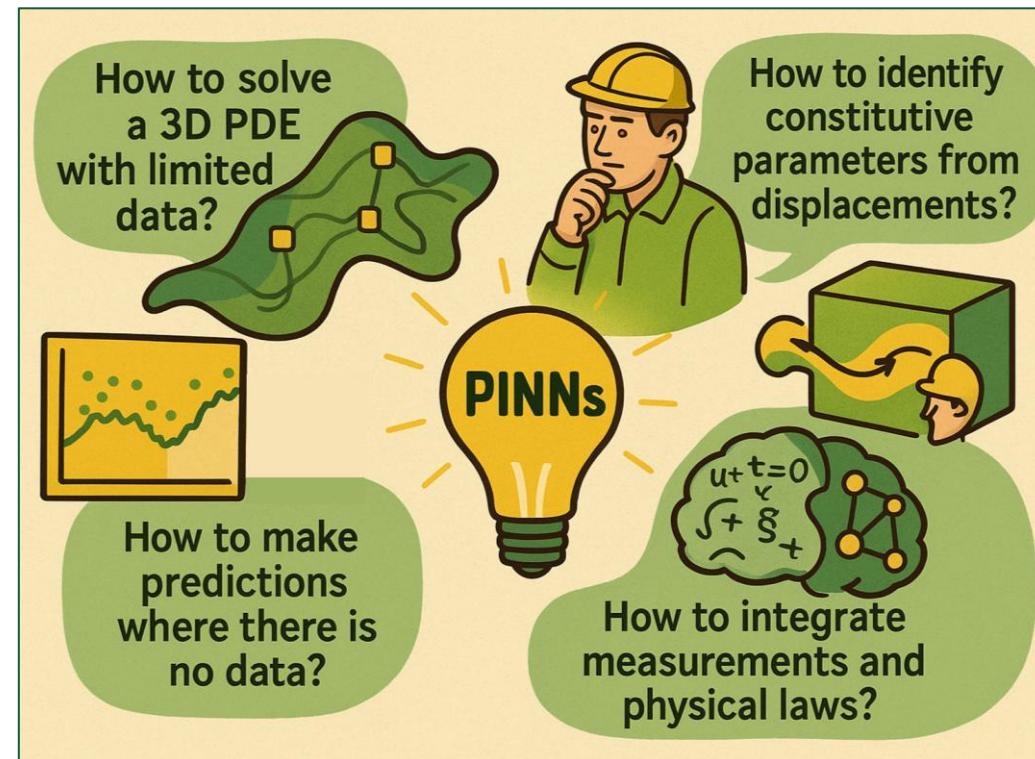
- Leis físicas conhecidas, mas os **dados experimentais** costumam ser **poucos**, **caros** ou **com ruído**.
- Muitas vezes queremos descobrir algo "**escondido**" (**problemas inversos**), como propriedades de um material sem medi-las diretamente.
- **Problemas mal definidos** aparecem com frequência e **são difíceis** de resolver com **métodos clássicos**.
- Métodos tradicionais, como **elementos finitos**, precisam de **malhas** que, em sua maioria, são difíceis de criar para geometrias complexas.
- Quando o problema tem **muitas variáveis ou dimensões**, os métodos numéricos tradicionais começam a falhar.
- É **difícil combinar dados reais** com **simulações clássicas**, porque os métodos numéricos não lidam bem com **medições incompletas** ou **irregulares**.



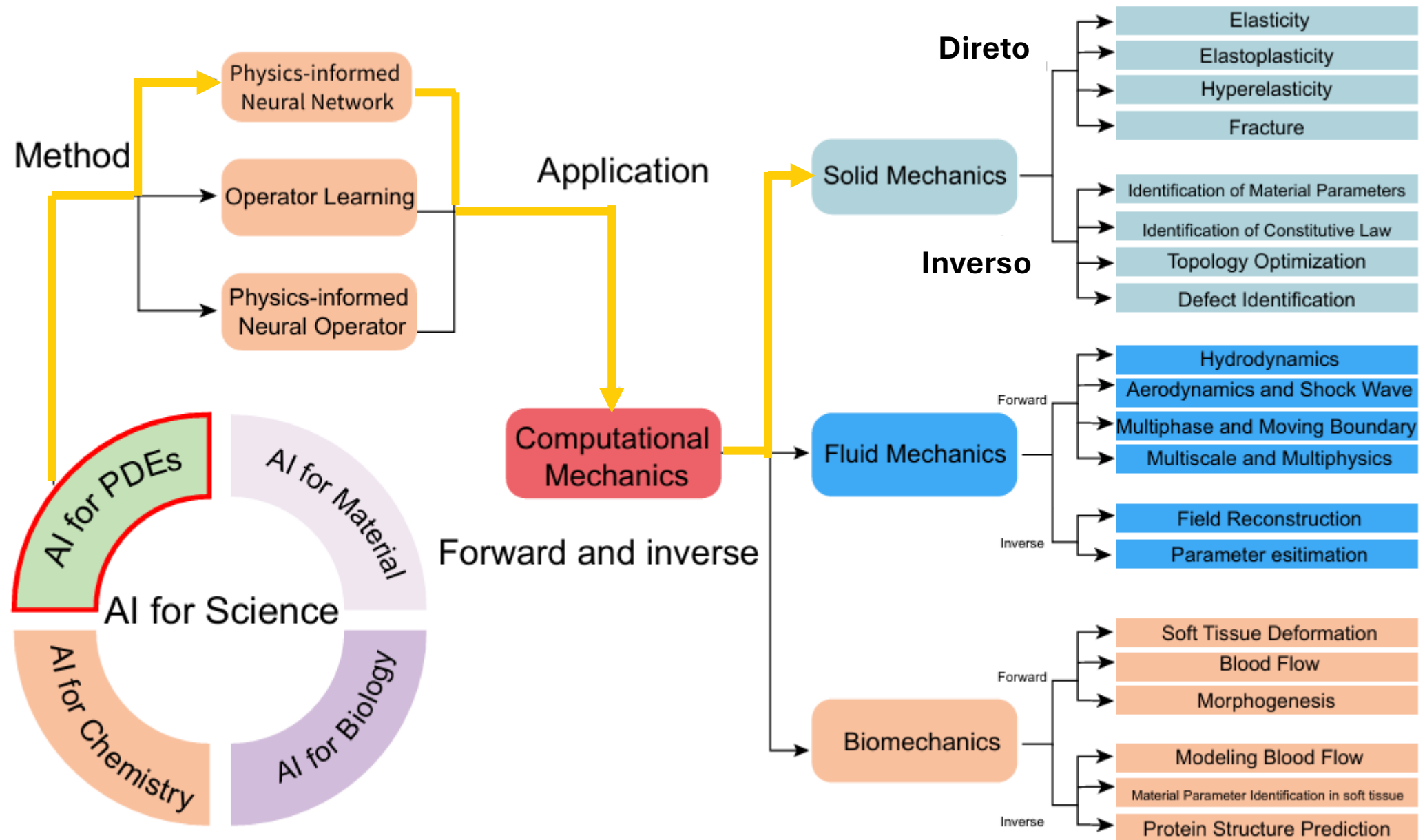
INTRODUÇÃO E MOTIVAÇÃO

Diante disso, surgem perguntas reais da engenharia e da ciência moderna:

- Como resolver uma EDP em um domínio 3D irregular com poucos dados experimentais?
- Como identificar parâmetros constitutivos de um material só a partir de medidas de deslocamento?
- Como fazer previsões confiáveis mesmo em regiões onde não há dados?
- Como integrar dados medidos e leis físicas em um mesmo modelo?



**Essas são as lacunas que motivaram o surgimento das
PINNs - Redes Neurais Informadas pela Física**





Revisão Inicial

Equações Diferenciais

Notação Indicial

Engenharia de Estruturas

Redes Neurais



Uma breve ideia das EDOs e EDPs

Equações Diferenciais Ordinárias (EDOs) e Parciais (EDPs)

EDO: envolvem derivadas em relação a uma única variável independente. $\frac{dy}{dx} = f(x)$

EDP: envolvem derivadas parciais com relação a duas ou mais variáveis independentes. $\frac{\partial u}{\partial t} = k \frac{\partial^2 u}{\partial x^2}$

Classificação

Linear: a função incógnita e suas derivadas aparecem apenas na primeira potência e não multiplicadas entre si. Ex.: $y'' + p(x)y' + q(x)y = g(x)$

Não-linear: contém termos com potências, produtos ou funções não-lineares da incógnita ou de suas derivadas.

Homogênea: se o termo independente for zero. Ex.: $y'' + p(x)y' + q(x)y = 0$

Não-homogênea: possui um termo independente não nulo. Ex.: $y'' + p(x)y' + q(x)y = g(x)$, com $g(x) \neq 0$

Uma breve ideia das EDOs e EDPs

Problema de Valor Inicial

Problema onde se busca uma solução para uma EDO ou EDP com condições especificadas em um ponto ou superfície inicial.

$$\text{Exemplo: } \frac{dy}{dx} = f(x, y) \text{ com } y(x_0) = y_0$$

Condições de Contorno: Neumann e Dirichlet

Dirichlet: especifica o valor da **função** na fronteira do domínio. Ex.: $u = g$ em $\partial\Omega$ (Γ).

Neumann: especifica o valor da **derivada normal** da função na fronteira (fluxo). Ex.: $\frac{\partial u}{\partial n} = h$ em $\partial\Omega$, onde $\frac{\partial}{\partial n}$ indica a derivada na direção normal à fronteira.

Uma breve ideia das EDOs e EDPs

Operador Nabla: ∇

- **Operação Gradiente:** Aplicado a função escalar – (Leva de escalar para vetor)

$$\nabla f(x, y, z) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z} \right)$$

- **Operação Divergente:** Aplicado a um campo vetorial $F = (F_x, F_y, F_z)$ – (Leva de vetor para escalar)

$$\nabla \cdot F = \frac{\partial F_x}{\partial x} + \frac{\partial F_y}{\partial y} + \frac{\partial F_z}{\partial z}$$

- **Operação Rotacional:** Aplicado a um campo vetorial – (Mantém-se vetor)

$$\nabla \times F = \left(\frac{\partial F_z}{\partial y} - \frac{\partial F_y}{\partial z}, \frac{\partial F_x}{\partial z} - \frac{\partial F_z}{\partial x}, \frac{\partial F_y}{\partial x} - \frac{\partial F_x}{\partial y} \right)$$

Notação Indicial

Em mecânica é comum usar notação indicial para expressar de forma compacta vetores, matrizes e sistemas de equações.

Nesse sentido entra o conceito de tensores de primeira e segunda ordem e algumas operações como contração simples e dupla, para mais detalhes consultar: [Elementos de Álgebra e Análise Tensorial para Mecânica](#)

Tensor de Ordem 1: $f_i = [f_1, f_2]^T$ para definir um vetor

Tensor de Ordem 2: $\sigma_{ij} = \begin{bmatrix} \sigma_{11} & \sigma_{12} \\ \sigma_{21} & \sigma_{22} \end{bmatrix}$ para definir uma matriz (ex. ao lado é uma matriz 2D).

$\sigma_{ij,j} = \nabla \cdot \sigma = \begin{bmatrix} \frac{\partial \sigma_{11}}{\partial x_1} + \frac{\partial \sigma_{12}}{\partial x_2} \\ \frac{\partial \sigma_{21}}{\partial x_1} + \frac{\partial \sigma_{22}}{\partial x_2} \end{bmatrix}$ é o divergente do tensor (derivada simbolizada por uma **vírgula**)

δ_{ij} é delta de Kronecker se $i = j$ então $\delta_{ii} = \delta_{jj} = 1$ e vale zero caso contrário.

Mecânica das Estruturas

Conceitos Essenciais

Dentro dos problemas que são abordados alguns conceitos são essenciais:

- Força, tensões, deformações, deslocamentos, deflexão, momento fletor;
- Parâmetros de Lamé, Módulo de Elasticidade, coeficiente de Poisson;
- Barra, viga, placas;

Disciplinas/Temas

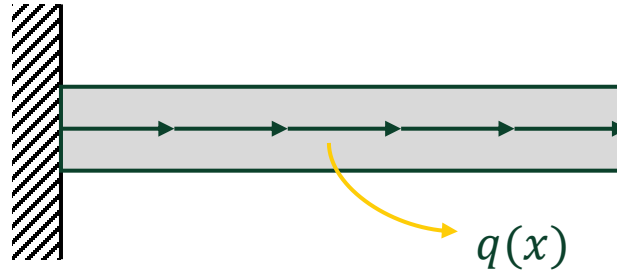
Algumas disciplinas ou temas que podem ser estudados antes de se aplicar as redes neurais:

- | | |
|--------------------------|--------------------------------|
| • Mecânica dos Sólidos | • Placas e Cascas |
| • Teoria da Elasticidade | • Vibrações |
| • Mecânica do Contínuo | • Método dos Elementos Finitos |

Mecânica das Estruturas

Barra Simples

$$\frac{d}{dx} \left(EA \frac{du}{dx} \right) = -q(x)$$

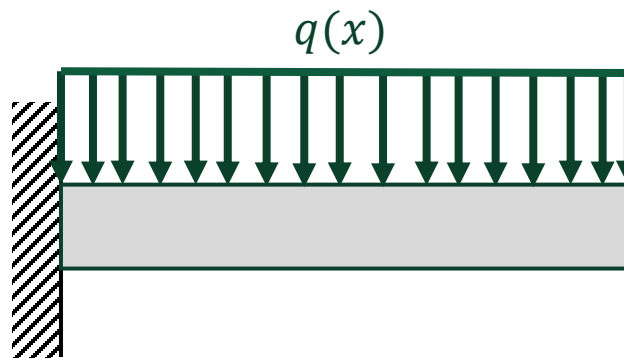


Viga de Euler-Bernoulli

$$EI \frac{d^4 y}{dx^4} = -q(x)$$

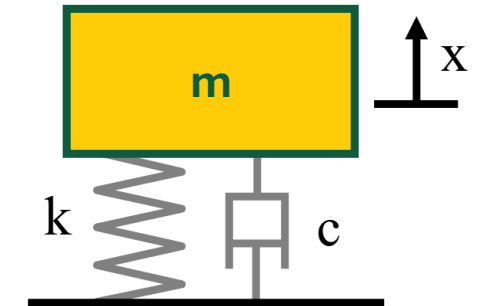
$$EI \frac{d^3 y}{dx^3} = V(x)$$

$$EI \frac{d^2 y}{dx^2} = M(x)$$



Massa-Mola-Amortecedor

$$M\ddot{u} + C\dot{u} + Ku = F$$

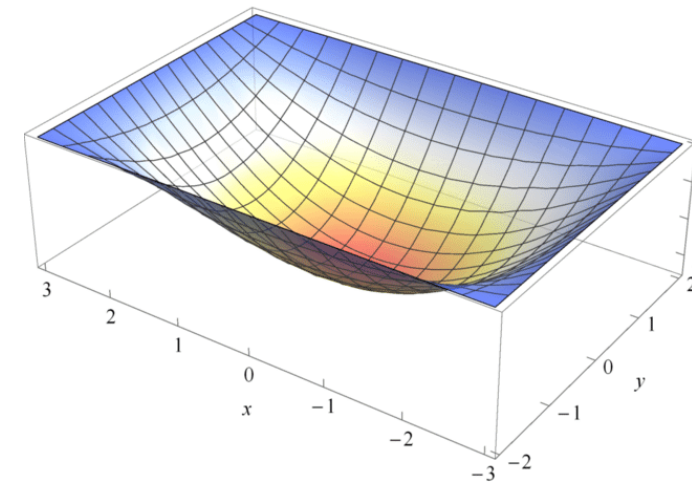
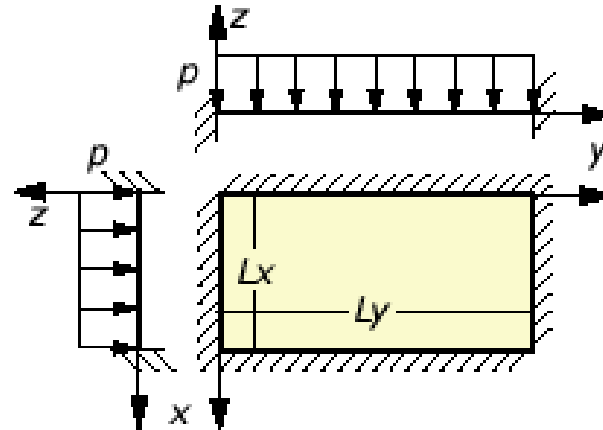


Mecânica das Estruturas

Deflexão em Placas

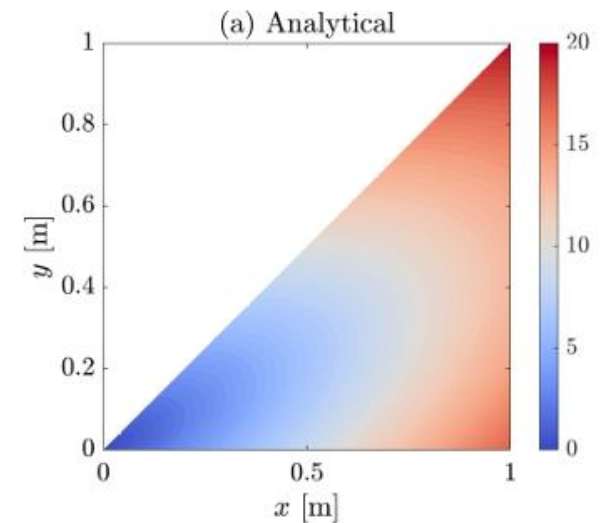
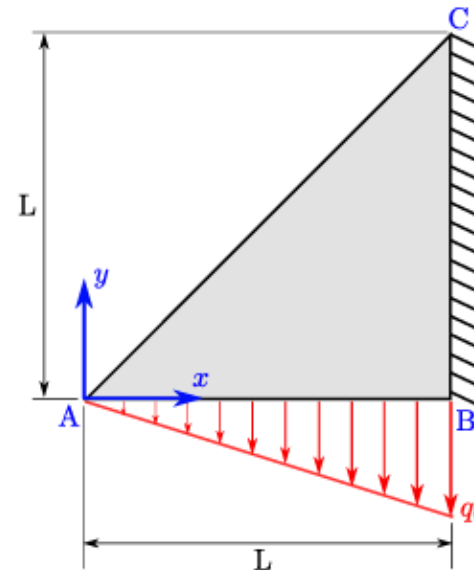
$$\frac{\partial^4 w}{\partial x^4} + 2 \frac{\partial^4 w}{\partial^2 x \partial^2 y} + \frac{\partial^4 w}{\partial^4 y} = \frac{p}{D} \quad \text{ou} \quad \nabla^4 w = \frac{p}{D}$$

$$D = \frac{Et^3}{12(1-\nu^2)}$$



Tensões em Placas (Função de Airy)

$$\frac{\partial^4 \phi}{\partial x^4} + 2 \frac{\partial^4 \phi}{\partial^2 x \partial^2 y} + \frac{\partial^4 \phi}{\partial^4 y} = 0$$



Mecânica das Estruturas

Elasticidade Linear

$$\begin{aligned}\nabla \sigma + f &= 0 \\ \sigma &= \lambda \delta_{ij} \varepsilon + 2\mu \varepsilon \\ \varepsilon &= \frac{1}{2} [\nabla \mathbf{u} + (\nabla \mathbf{u})^T]\end{aligned}$$

Parâmetros de Lamé:

$$\begin{aligned}\lambda &= \frac{E\nu}{(1+\nu)(1-2\nu)} \\ \mu &= \frac{E}{2(1+\nu)}\end{aligned}$$

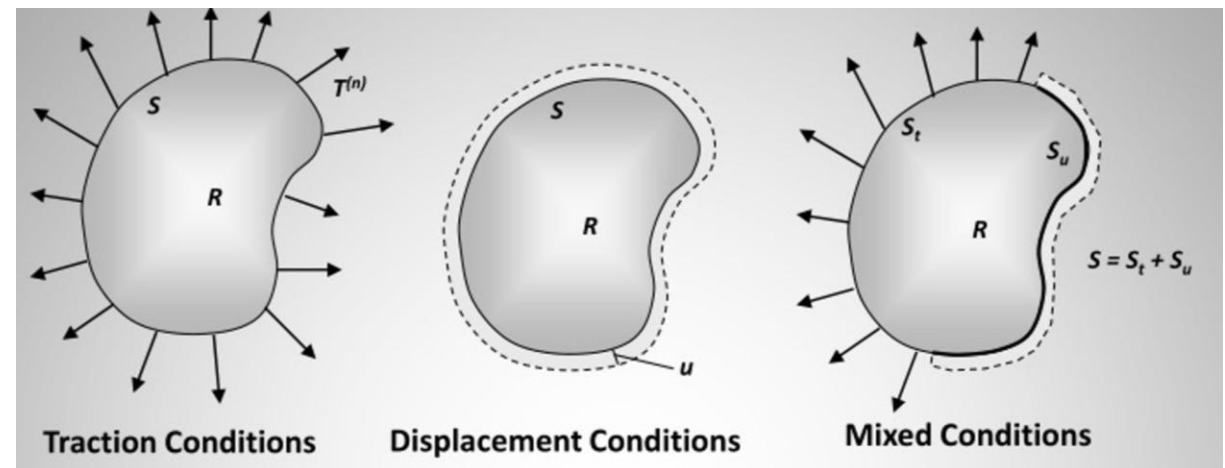
Condições Iniciais e de Contorno

Deslocamentos IC: $u(x, 0) = u_0(x)$ *em* Ω

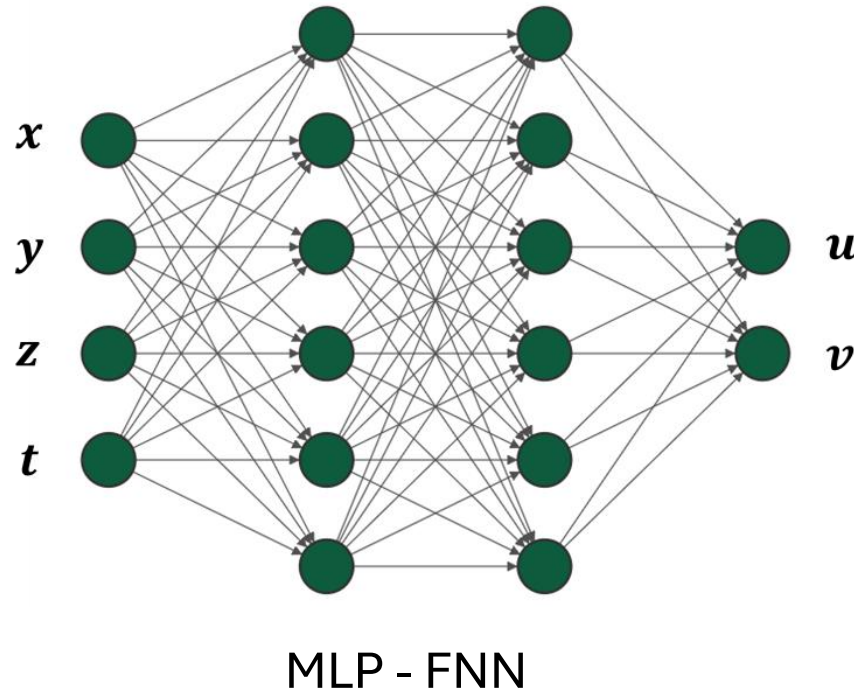
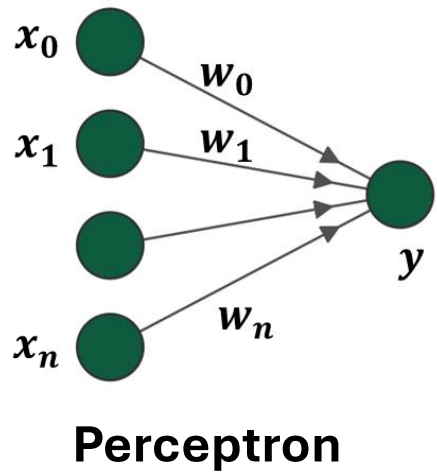
Velocidades IC: $\dot{u}(x, 0) = v_0(x)$ *em* Ω

Deslocamentos BC: $u = \bar{u}$ *em* Γ

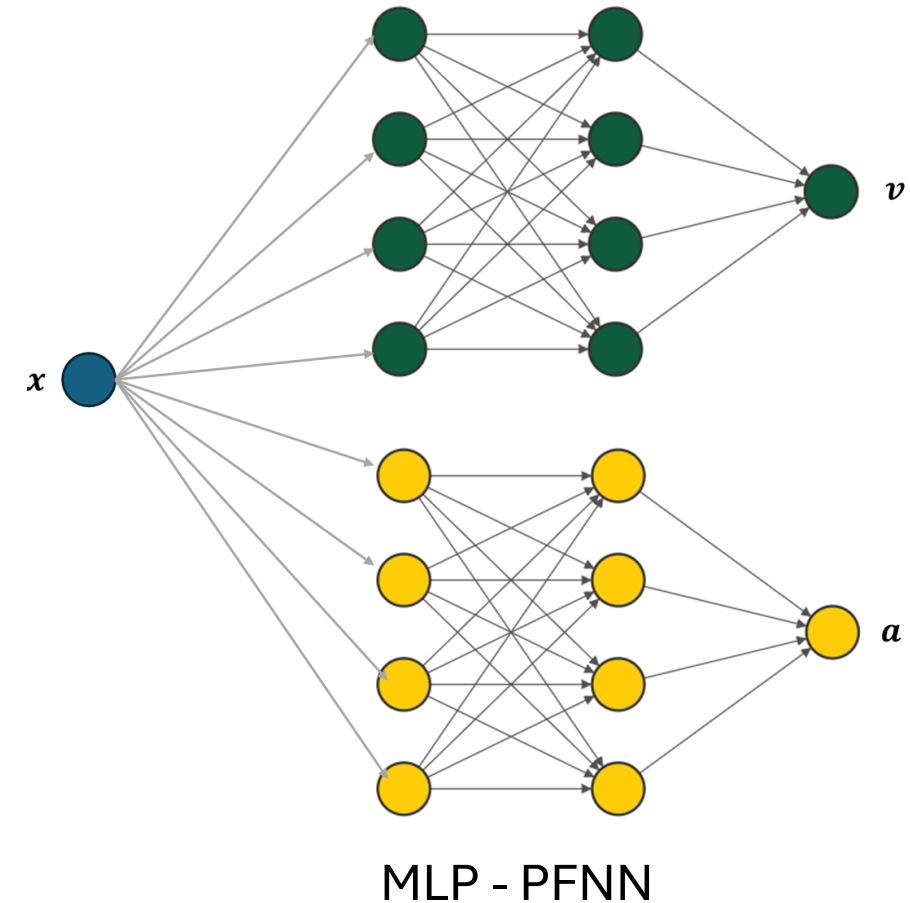
Forças BC: $t = \bar{t} = \sigma n$ *em* Γ



Uma breve ideia das NNs - Arquitetura

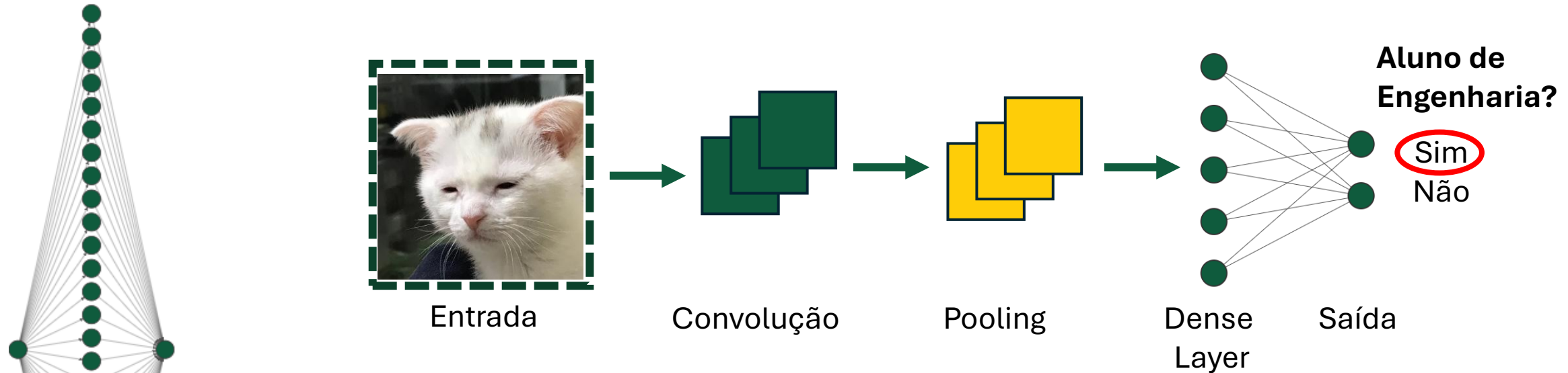


Fully-connected neural network



Parallel fully-connected network

Uma breve ideia das NNs - Arquitetura



Redes Neurais Convolucionais

Convolutional Neural Networks from Scratch

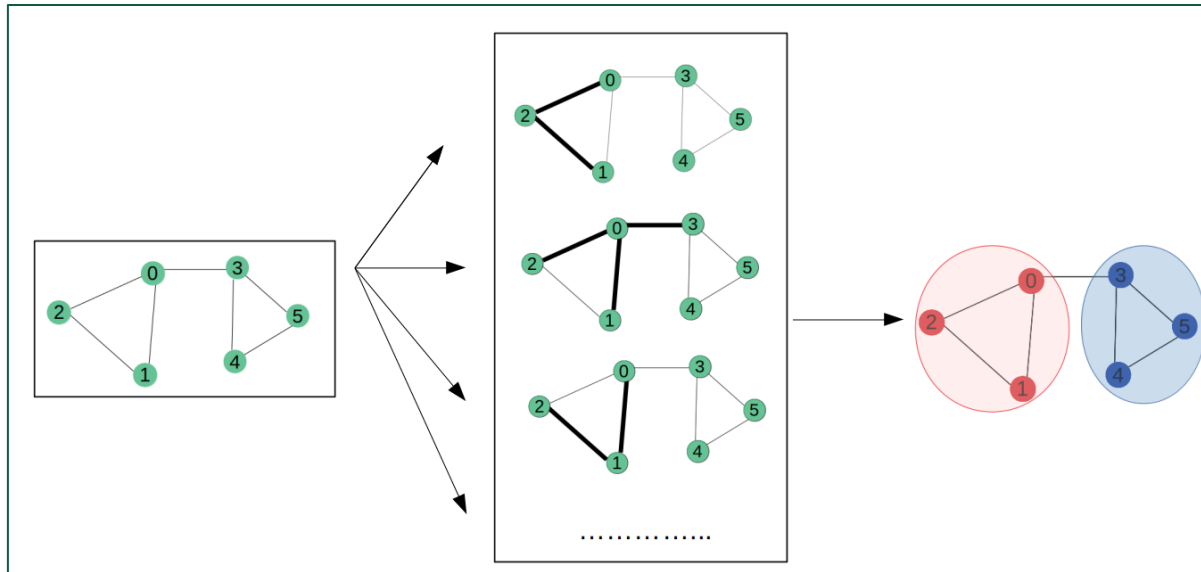
<https://youtu.be/jDe5BAsT2-Y?si=tySCQonvLXk9Cvfm>

A máquina de Aprendizado Extremo:

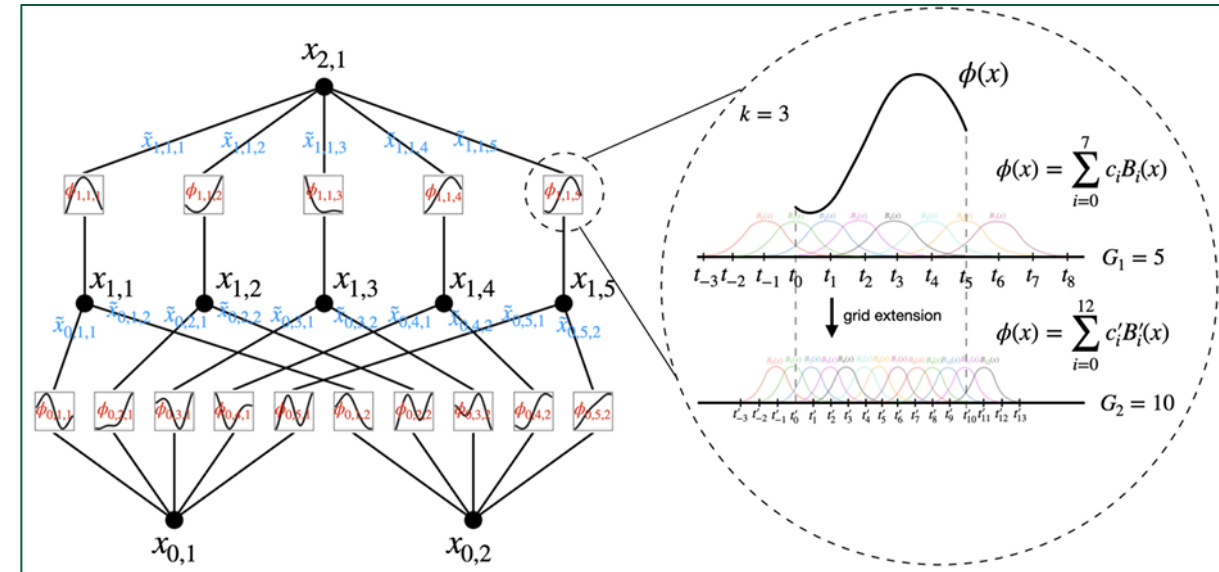
<https://computacaointeligente.com.br/algoritmos/maquina-de-aprendizado-extremo/>

Extreme Learning Machine

Uma breve ideia das NNs - Arquitetura



Graph Convolution Network



Kolmogorov-Arnold Network

Graph Convolutional Network

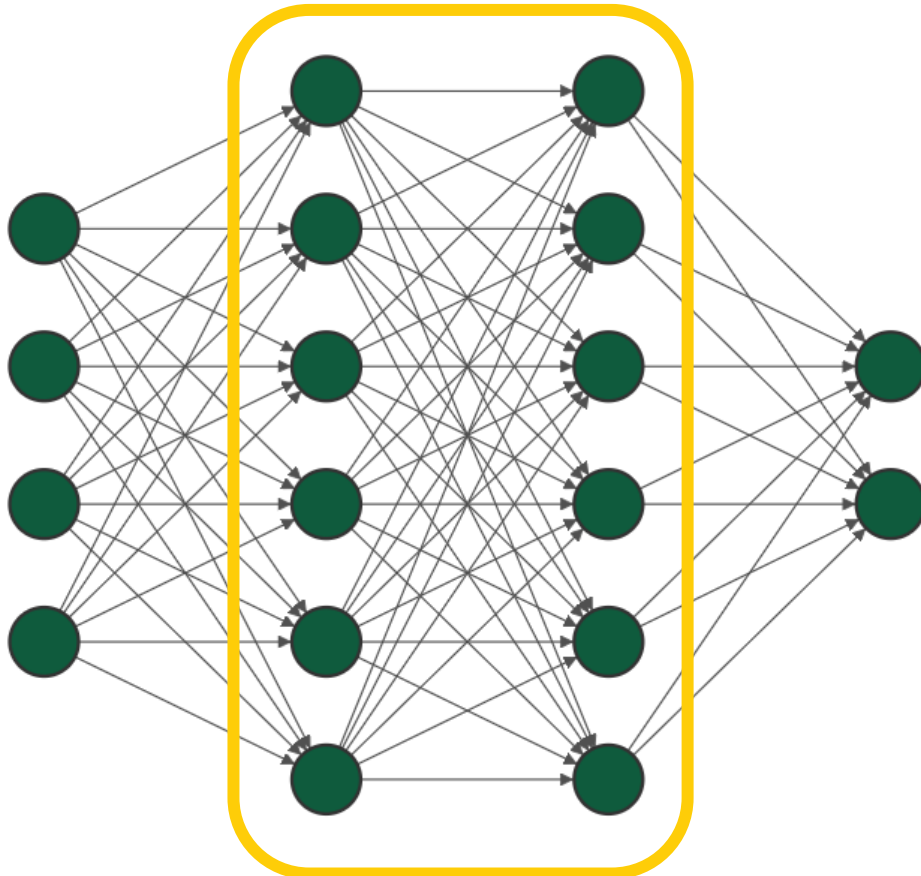
<https://paperswithcode.com/method/gcn>

KAN: Kolmogorov-Arnold Networks

<https://arxiv.org/abs/2404.19756>

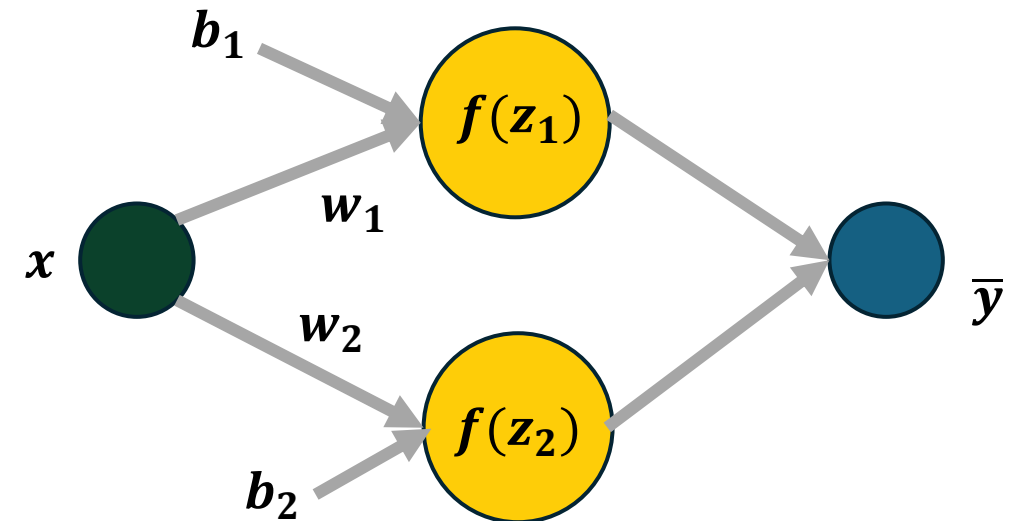
Uma breve ideia das NNs - Parâmetros

Camadas Ocultas



Hidden Layers

Pesos (w) e Viés (b)



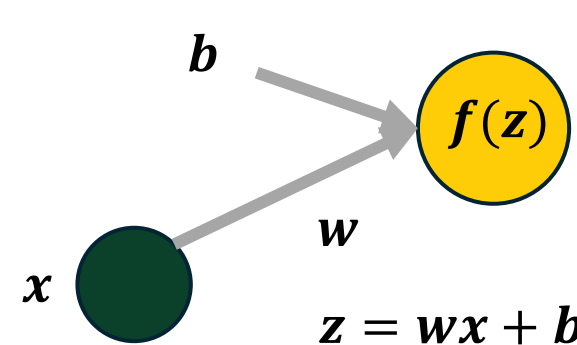
$$z_1 = w_1 x + b_1$$

$$z_2 = w_2 x + b_2$$

Função de Ativação (f)

Uma breve ideia das NNs - Parâmetros

Função de Ativação (f)



f {
Sigmoide
Tangente Hiperbólica

ReLU $\max(0, x)$	GELU $\frac{x}{2} \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + ax^3) \right) \right)$	PReLU $\max(0, x)$
ELU $\begin{cases} x & \text{if } x > 0 \\ \alpha(x \exp x - 1) & \text{if } x < 0 \end{cases}$	Swish $\frac{x}{1 + \exp -x}$	SELU $\alpha(\max(0, x) + \min(0, \beta(\exp x - 1)))$
SoftPlus $\frac{1}{\beta} \log(1 + \exp(\beta x))$	Mish $x \tanh \left(\frac{1}{\beta} \log(1 + \exp(\beta x)) \right)$	RReLU $\begin{cases} x & \text{if } x \geq 0 \\ ax & \text{if } x < 0 \text{ with } a \sim \mathcal{R}(l, u) \end{cases}$

HardSwish $\begin{cases} 0 & \text{if } x \leq -3 \\ x & \text{if } x \geq 3 \\ x(x+3)/6 & \text{otherwise} \end{cases}$	Sigmoid $\frac{1}{1 + \exp(-x)}$	SoftSign $\frac{x}{1 + x }$
Tanh $\tanh(x)$	Hard tanh $\begin{cases} a & \text{if } x \geq a \\ b & \text{if } x \leq b \\ x & \text{otherwise} \end{cases}$	Hard Sigmoid $\begin{cases} 0 & \text{if } x \leq -3 \\ 1 & \text{if } x \geq 3 \\ x/6 + 1/2 & \text{otherwise} \end{cases}$
Tanh Shrink $x - \tanh(x)$	Soft Shrink $\begin{cases} x - \lambda & \text{if } x > \lambda \\ x + \lambda & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$	Hard Shrink $\begin{cases} x & \text{if } x > \lambda \\ x & \text{if } x < -\lambda \\ 0 & \text{otherwise} \end{cases}$

Uma breve ideia das NNs - Parâmetros

Inicializador de Pesos **Glorot (Xavier) ou He**:

A maneira como algoritmo define o intervalo no qual os pesos devem ser iniciados.

Otimizadores Adam ou BFGS: Forma como a rede encontra os melhores pesos e vieses, normalmente utiliza-se **Métodos de Gradiente + Backpropagation + Diferenciação Automática**. Veja: [Playlist StatQuest](#)

Épocas: Quantidade de iterações para se obter a convergência dos resultados.

Função de Perda (Loss Function): Função a ser minimizada pelos otimizadores ao longo das épocas, normalmente definida pela função **Mean Square Error (MSE)** ou **Norma L^2**

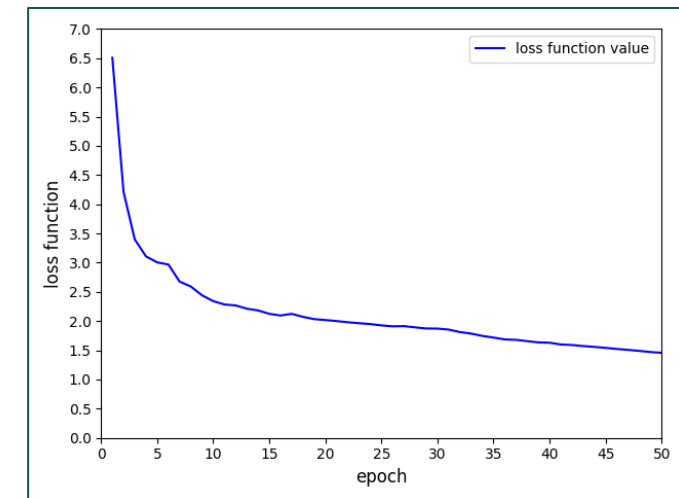
○ Xavier Normal Initialization

$$W \sim N(0, Var(W))$$

$$Var(W) = \sqrt{\frac{2}{n_{in} + n_{out}}}$$

○ Xavier Uniform Initialization

$$W \sim U\left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, +\sqrt{\frac{6}{n_{in} + n_{out}}}\right)$$



$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 \quad |\mathbf{x}| = \sqrt{\sum_{k=1}^n |x_k|^2},$$

Uma breve ideia das NNs - Bibliotecas



Sites das Bibliotecas:

- <https://numpy.org/>
- <https://pytorch.org/>
- <https://www.tensorflow.org/?hl=pt-br>
- <https://scikit-learn.org/stable/>

Os códigos serão desenvolvidos no **Google Colab**:

- Basta ter uma conta no GoogleDrive
- Acesso a CPU e GPU
- Não precisa instalar nada na sua máquina



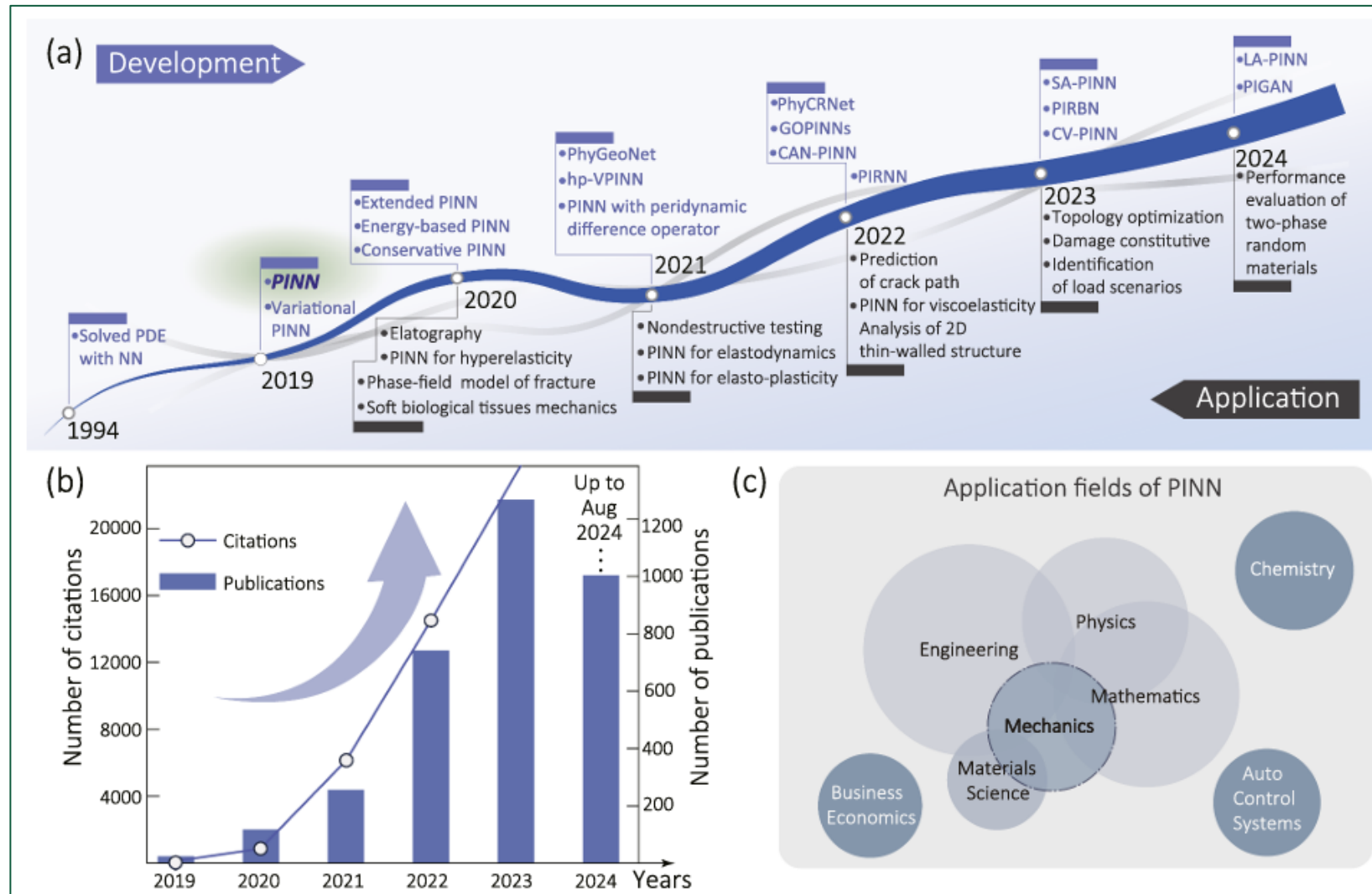
Focaremos em desenvolver os códigos com o **PyTorch**



Redes Neurais e EDs



Redes Neurais e Equações Diferenciais



Hu et al (2024)

Redes Neurais e Equações Diferenciais

Surgimento das estratégias:

Physics Informed Neural Networks (PINN)

- Forma Forte: PINN tradicional
- Forma Energética: Deep Energy Method (DEM)

Operator Learning

- DeepONet
- FNO: Fourier Neural Operator

Physics Informed Neural Operator (PINO)



Physics Informed Neural Networks

O que são PINNs?

Parâmetros, Definições e Particularidades

Desafios e Limitações

Afinal, o que são PINNs?

Considere um **operador diferencial** genérico $\mathcal{N}[u(x, t)] = 0$

Uma rede neural tradicional nos dá: $f(x, t, \mathbf{W}) \approx \hat{u}$

Podemos aplicar o mesmo operador sobre a rede

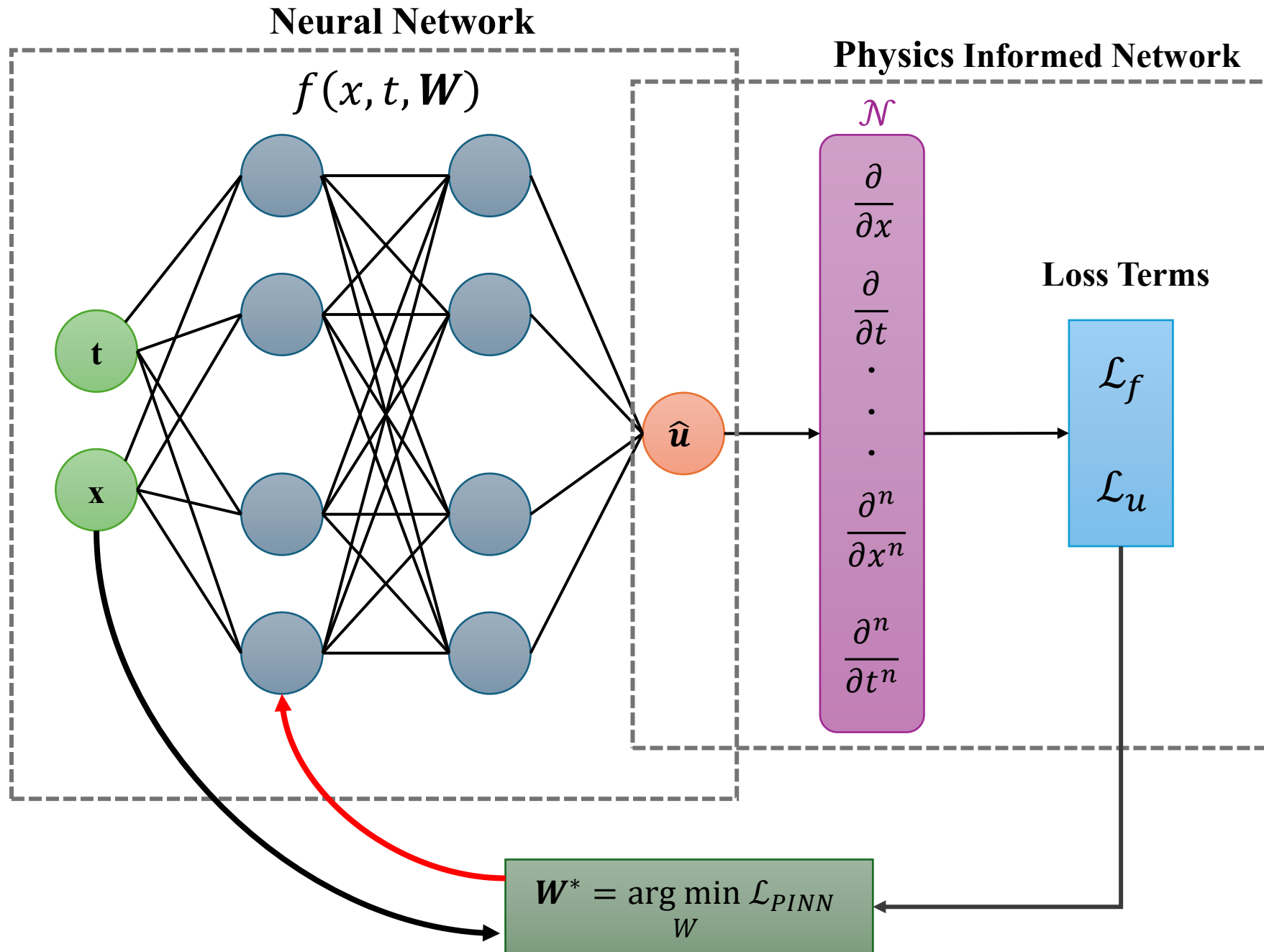
$$\mathcal{N}[\hat{u}] \approx \mathcal{N}[u(x, t)]$$

E atribuir **funções de perda** tanto para esse **operador**, quanto para **condições iniciais** ou de **contorno** do problema

$$\mathcal{L}_f = \frac{1}{N_f} \sum_{i=1}^{N_f} |\mathcal{N}(\hat{u}(x_i, t_i))|^2, \quad \mathcal{L}_u = \frac{1}{N_u} \sum_{i=1}^{N_u} |u_i - \hat{u}(x_i, t_i)|^2,$$

E assim temos uma função de perda total para a PINN, que deve ser minimizada

$$\mathcal{L}_{PINN} = \mathcal{L}_f + \mathcal{L}_u$$



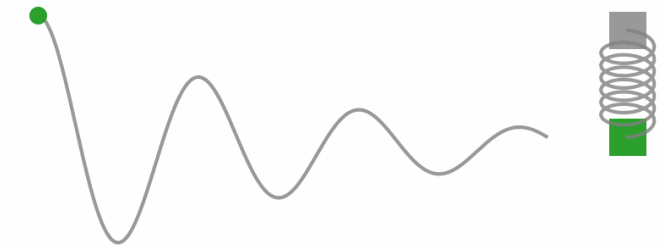
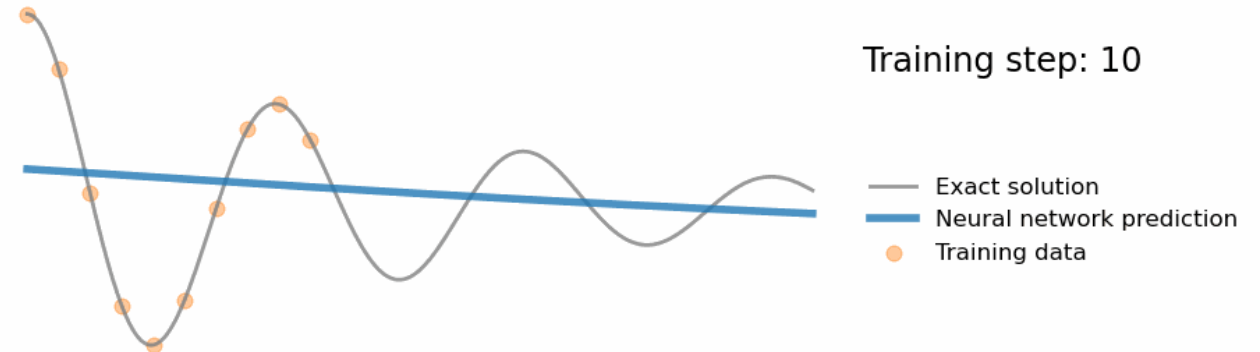
Afinal, o que são PINNs?

Exemplo com base em [1]

Rede Neural Tradicional:

- Fornecer dados da curva original, no qual o ajuste (fitting) é feito somente nesses pontos
- Ter a solução via amostragem, solução analítica ou Runge-Kutta
- Gerar dados de entrada (x) e saída (u)
- A rede será treinada para achar o melhor ajuste para os x_i e $u(x_i)$ **fornecidos** retornando u_{NN}

$$\min \frac{1}{N} \sum_i^N (u_{NN}(x_i) - u(x_i))^2$$

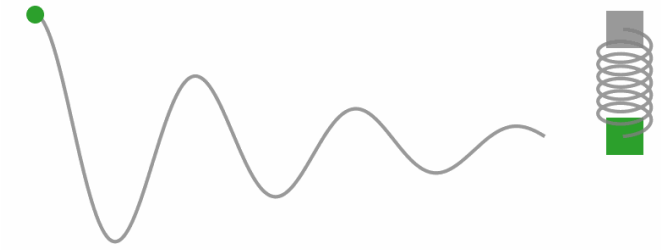


$$M\ddot{u} + C\dot{u} + Ku = 0$$

Afinal, o que são PINNs?

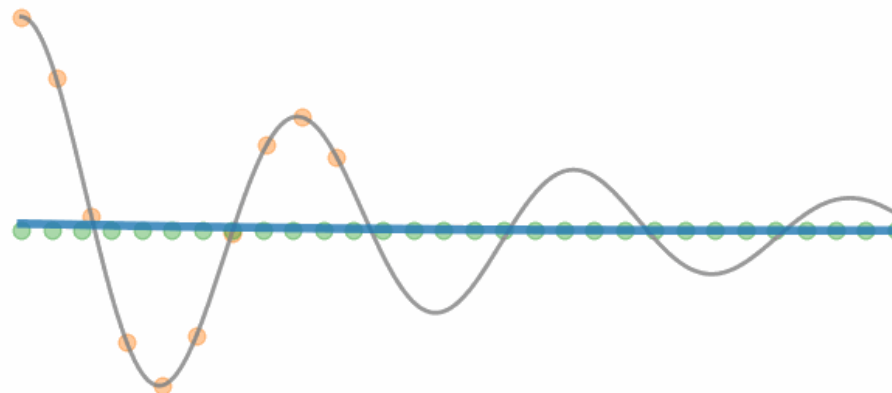
Redes Informadas por Física

- Fornecer a equação diferencial e calcular em pontos do domínio (x_c)
- Fornecer as condições de contorno: $u(0) = u_0$, $\dot{u}(0) = v_0$
- Fornecer dados da curva original (opcional)



$$M\ddot{u} + C\dot{u} + Ku = 0$$

$$\min \underbrace{\frac{1}{N_c} \sum_i^{N_c} (M\ddot{u}_{NN}(x_c) + C\dot{u}_{NN}(x_c) + Ku_{NN}(x_c))^2}_{\mathcal{L}_{PDE}} + \underbrace{\frac{1}{N_{ib}} \sum_i^{N_{ib}} (u_{NN}(x_{ib}) - u(x_{ib}))^2}_{\mathcal{L}_{ICBC}} + \underbrace{\frac{1}{N} \sum_i^N (u_{NN}(x_i) - u(x_i))^2}_{\mathcal{L}_{Dados}}$$



Training step: 150

- Exact solution
- Neural network prediction
- Training data
- Physics loss training locations

Afinal, o que são PINNs?

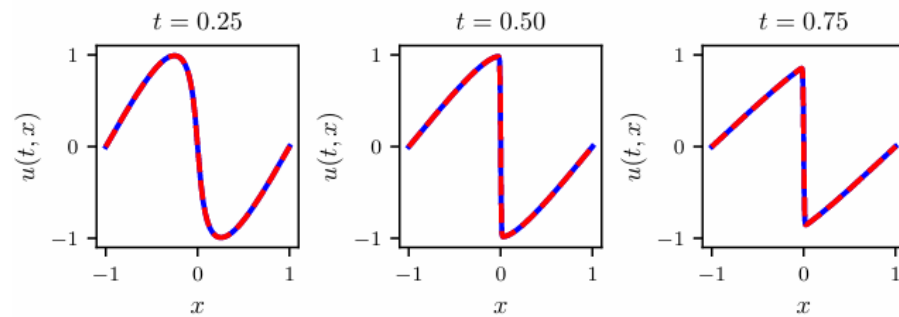
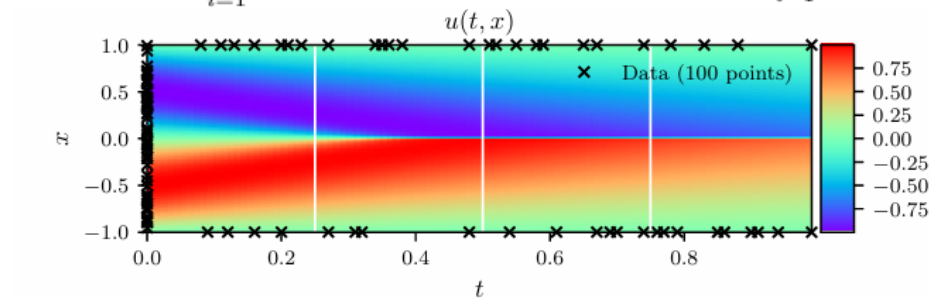
Equação de Burgers 1D no espaço

$$u_t + uu_x - (0.01/\pi)u_{xx} = 0, \quad x \in [-1, 1], \quad t \in [0, 1],$$

$$u(0, x) = -\sin(\pi x),$$

$$u(t, -1) = u(t, 1) = 0.$$

$$MSE_u = \frac{1}{N_u} \sum_{i=1}^{N_u} |u(t_u^i, x_u^i) - u^i|^2, \quad MSE_f = \frac{1}{N_f} \sum_{i=1}^{N_f} |f(t_f^i, x_f^i)|^2.$$



— Exact - - - Prediction

Raissi (2017)

Problema de Elasticidade Linear

$$\nabla \cdot \sigma + F = \rho \ddot{u} \quad \varepsilon = \frac{1}{2} [\nabla u + (\nabla u)^T]$$

$$\sigma = C : \varepsilon$$

$$L = L_d + L_g =$$

$$\|u_x - u_x^*\| + \|u_y - u_y^*\| + \|\sigma_{xx} - \sigma_{xx}^*\| + \|\sigma_{yy} - \sigma_{yy}^*\| + \|\sigma_{xy} - \sigma_{xy}^*\| +$$

$$\left\| \frac{\partial \sigma_{xx}}{\partial x} + \frac{\partial \sigma_{xy}}{\partial y} \right\| + \left\| \frac{\partial \sigma_{yy}}{\partial y} + \frac{\partial \sigma_{xy}}{\partial x} \right\| +$$

$$\|(\lambda + 2\mu)\varepsilon_{xx} + \lambda\varepsilon_{yy} - \sigma_{xx}\| +$$

$$\|(\lambda + 2\mu)\varepsilon_{yy} + \lambda\varepsilon_{xx} - \sigma_{yy}\| + \|2\mu\varepsilon_{xy} - \sigma_{xy}\|$$

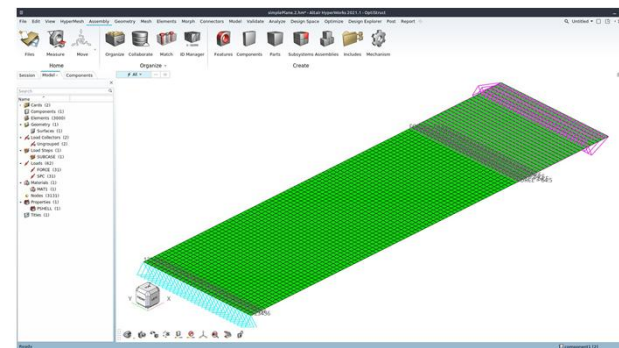


Figure 6: The problem setup in HyperWorks.

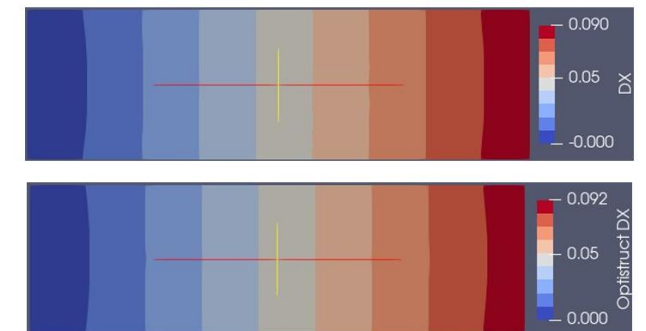


Figure 9: The DX displacement predicted by the NN (top) and the results from the OptiStruct solver (bottom).

Nentidis (2022)

Parâmetros, Definições e Particularidades

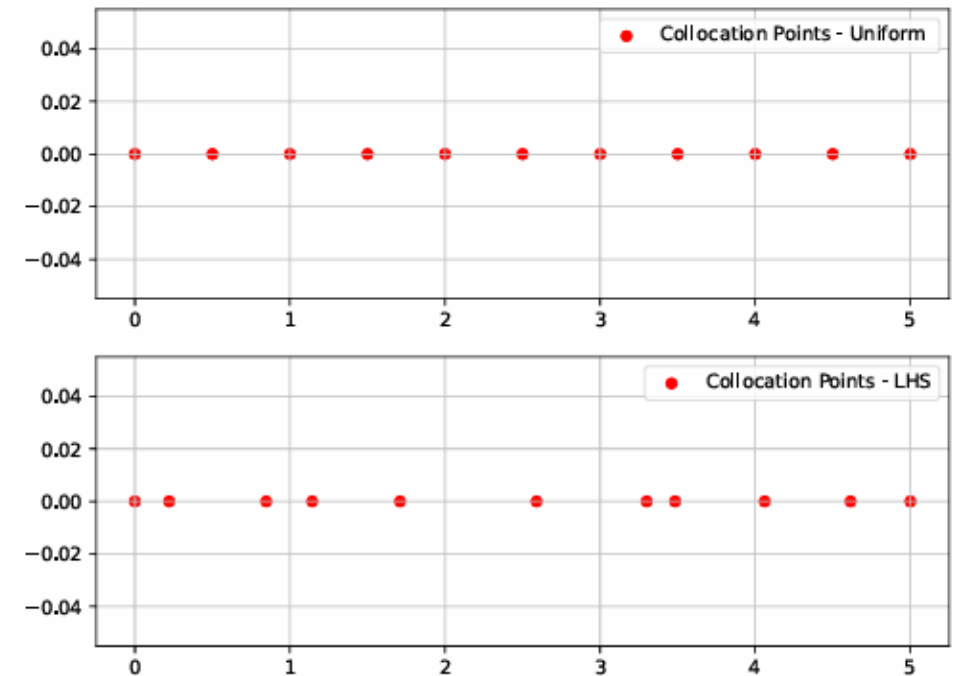
Pontos de colocação: São pontos espalhados pelo domínio e que são introduzidos na Loss durante a minimização.

- Reduzir a quantidade de pontos, reduz o custo computacional;
- **Soluções Triviais:** A PINN retorna um resultado nulo;
- Usar espaçamentos especiais:
 - Distribuição Latin Hypercube Sample
 - Distribuição Aleatória
 - Distribuição de Sobol

Qtd. Camadas Escondidas: 2 à 6 camadas

Qtd. Neurônios: 5 à 64

Função de Ativação: Sigmoid e Tangente Hiperbólica



Paulo Silva (2025) – Não Publicado

Leiteritz (2021)

How to Avoid Trivial Solutions in Physics-Informed Neural Networks

Cuomo (2022)

Scientific Machine Learning Through Physics-Informed Neural Networks: Where we are and What's Next

Vasily (2024)



ARE TWO HIDDEN LAYERS STILL ENOUGH FOR THE PHYSICS-INFORMED NEURAL NETWORKS?

Parâmetros, Definições e Particularidades ICBC* – Fracas (Soft/Weak) e Fortes (Hard/Strong)

Existem duas formas de garantir que as condições de contorno (CCs) sejam atendidas;

Soft

Características

As CCs são adicionadas na Loss
A rede neural é livre porém penalizada
Versão mais utilizada, principalmente quando a imposição é complexa

Vantagens

Fácil Implementar
Flexível para problemas complicados

Desvantagens

Imprecisão
Necessidade de ajustes finos

Hard

Características

As CCs são adicionadas na arquitetura da solução
A rede é construída para satisfazer a CC
Normalmente modifica a solução via funções multiplicadoras

Vantagens

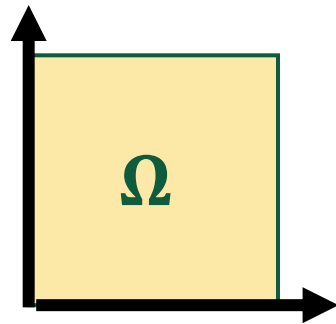
Garante que as CCs sejam satisfeitas
Pode melhorar a precisão e a eficiência do treino

Desvantagens

Mais difícil de implementar
Pode restringir a flexibilidade da rede

Parâmetros, Definições e Particularidades

Suporte Compacto: Termo utilizado para “fechar” e “delimitar” o domínio de estudo, fazendo com que a região seja definida em condições que retornam resultados nulos, muito utilizado na definição das **hard constraints**. Veja o exemplo:



$$\Omega = [0 \ 1] \times [0 \ 1]$$

Suporte compacto com a função: $x(1 - x)$

Batch size (Tamanho do Lote): define quantos exemplos do conjunto de dados são processados antes de atualizar os pesos do modelo. O *batch size* determina:

- Quantos dados são usados por vez para calcular os gradientes.
- Com que frequência os pesos do modelo são atualizados.
- **Valores comuns:** 16, 32, 64, 128

Desafios e Limitações

Apesar de promissoras, as PINNs ainda enfrentam dificuldades técnicas, como:

- **Viés espectral:** redes tendem a aprender primeiro as frequências baixas.
- **Dificuldade de treinamento:** as funções de perda podem ser mal condicionadas.
- Sensibilidade à escolha dos pesos entre os termos da perda (dados vs. física).
- **Escalabilidade** para problemas muito grandes ou com soluções de comportamento local complexo.



Bibliotecas

DeepXDE

SciANN

PINN no MATLAB



Bibliotecas de PINN

Name	Address of the open-source code	Language	Refs.
NeuralPDE	https://github.com/SciML/NeuralPDE.jl.git	Julia	[129]
Neural Solvers	https://github.com/Photon-AI-Research/NeuralSolvers.git	Python	[130]
IDRLnet	https://github.com/idrl-lab/idrlnet.git	Python	[131]
PINA	https://github.com/mathLab/PINA.git	Python	[132]
TensorDiffEq	https://github.com/SciML/TensorFlowDiffEq.jl.git	Julia	[133]
DeepINN	https://github.com/praksharma/DeepINN.git	Jupyter Notebook	[134]
NeuralUQ	https://github.com/Crunch-UQ4MI/neuraluq.git	Python	[135]
Neurodiffeq	https://github.com/NeuroDiffGym/neurodiffeq.git	Python	[136]
DeepXDE	https://github.com/lululxvi/deepxde	Python	[137]
SciANN	https://github.com/sciann/sciann.git	Python	[138]
SimNet	https://developer.nvidia.com/simnet	Python	[139]
ADCME	https://github.com/kailaix/ADCME.jl.git	Julia	[140]
PDEBench	https://github.com/pdebench/PDEBench.git	Jupyter Notebook	[141]
PINNs-Torch	https://github.com/rezaakb/pinns-torch.git	Python	[142]
Neural Eikonal solver	https://github.com/sgrubas/NES.git	Jupyter Notebook	[143]
PinnDE	https://github.com/JB55Matthews/PinnDE	Python	[144]
MLniCS	https://github.com/MLniCS/MLniCS	Jupyter Notebook	[145]
PINNLib	https://github.com/m-raj/PINNLib	Python	[146]
Porch	https://github.com/leiterrl/porch	Python	
Jinns	https://github.com/mia-jinns/jinns	Jupyter Notebook	

Hu etal (2024)

DeepXDE

DEEPXDE: A DEEP LEARNING LIBRARY FOR SOLVING DIFFERENTIAL EQUATIONS

LU LU*, XUHUI MENG*, ZHIPING MAO*, AND GEORGE EM KARNIADAKIS*†

<https://doi.org/10.48550/arXiv.1907.04502>

Múltiplos backends: PyTorch, TensorFlow, JAX e PaddlePaddle.

<https://deepxde.readthedocs.io/en/latest/index.html>

Search docs

Install and Setup
Demos of Function Approximation
Demos of Forward Problems
Demos of Inverse Problems
Demos of Operator Learning
Parallel Training
FAQ
Research

A simple ODE system

Problem setup

We will solve a simple ODE system:

$$\frac{dy_1}{dt} = y_2, \quad \frac{dy_2}{dt} = -y_1, \quad \text{where } t \in [0, 10],$$

with the initial conditions

$$y_1(0) = 0, \quad y_2(0) = 1.$$

The reference solution is $y_1 = \sin(t)$, $y_2 = \cos(t)$.

Implementation

This description goes through the implementation of a solver for the above ODE system step-by-step.

First, the DeepXDE and NumPy (`np`) modules are imported:

```
import deepxde as dde
import numpy as np
```

FAQ
Research
Cite DeepXDE
The Team
API
deepxde
deepxde.data
deepxde.geometry
deepxde.gradients
deepxde.icbc
deepxde.nn
deepxde.nn.jax
deepxde.nn.paddle
deepxde.nn.pytorch
deepxde.nn.tensorflow
deepxde.nn.tensorflow_compat_v1
deepxde.optimizers
deepxde.optimizers.pytorch

```
geom = dde.geometry.TimeDomain(0, 10)
```

Next, we express the ODE system:

```
def ode_system(x, y):
    y1, y2 = y[:, 0:1], y[:, 1:]
    dy1_x = dde.grad.jacobian(y, x, i=0)
    dy2_x = dde.grad.jacobian(y, x, i=1)
    return [dy1_x - y2, dy2_x + y1]
```

The first argument to `ode_system` is the network input, i.e., the t -coordinate, and here we represent it as `x`. The second argument to `ode_system` is the network output, which is a 2-dimensional vector where the first component (`y[:, 0:1]`) is y_1 and the second component (`y[:, 1:]`) is y_2 .

Next, we consider the initial condition. We need to implement a function, which should return `True` for points inside the subdomain and `False` for the points outside. In our case, the point t of the initial condition is $t = 0$. (Note that because of rounding-off errors, it is often wise to use `dde.utils.isclose` to test whether two floating point values are equivalent.)

```
def boundary(x, on_initial):
    return dde.utils.isclose(x[0], 0)
```

YouTube: <https://youtube.com/playlist?list=PLM7DTyYjZGuLmg3f6j40fEF18jyQmYsC2>

SciANN

Múltiplos backends: PyTorch, TensorFlow e Keras

<https://www.sciann.com/>



<https://github.com/ehsanhaghighat/sciann-applications>

SciANN-BurgersEquation	Minor update on .eval	5 years ago
SciANN-ConstitutiveModeling	Constitutive characterization codes	3 years ago
SciANN-DataGenerator	minor	4 years ago
SciANN-Elasticity	Elasticity example from DesignSafe seminar	4 years ago
SciANN-ElastoPlasticity	type	5 years ago
SciANN-LaplaceEq-Forward	Laplace PDE example for Design Safe seminar	4 years ago
SciANN-Linear-Quadratic-Regression	minor update	5 years ago
SciANN-NN-Regression	Illustrative example of fitting using ANN	5 years ago
SciANN-NavierStokes	Navier-Stokes with SciANN	5 years ago
SciANN-PoroElasticity-Inverse	inverse poroelasticity	3 years ago
SciANN-PoroElasticity	Update README.md	3 years ago
SciANN-SolidMechanics-BCs	Compatibility with V>0.6	4 years ago
SciANN-SolidMechanics	compatibility with V>0.6	4 years ago
SciANN-Vibrations	SciANN-Vibrations	4 years ago

SciANN: A Keras/TensorFlow wrapper for scientific computations and physics-informed deep learning using artificial neural networks

Ehsan Haghighat^{*}, Ruben Juanes

<https://doi.org/10.1016/j.cma.2020.113552>

A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics

Ehsan Haghighat^a, Maziar Raissi^b, Adrian Moure^c, Hector Gomez^c, Ruben Juanes^{a,*}

<https://doi.org/10.1016/j.cma.2021.113741>

sciann-applications / SciANN-BurgersEquation / SciANN-BurgersEquation.ipynb
↑ Top

Preview Code Blame 423 lines (423 loc) · 29.7 KB Code 55% faster with GitHub Copilot
Raw Copy Download Edit

The governing equation is:

$$u_t + uu_x - (0.01/\pi)u_{xx} = 0, \quad t \in [0, 1], \quad x \in [-1, 1]$$

subjected to

$$u(t=0, x) = -\sin(\pi x), \quad u(t, x = \pm 1) = 0$$

The objective is to solve this equation with PINN.

Step 1: Setting up the neural network

The first step is to construct the approximation space.

The independent variables are: (t, x) The field variable is: $u(t, x)$

The neural network approximation is defined as:

$$\hat{u} : (t, x) \mapsto \mathcal{N}_u(t, x; \mathbf{W}, b)$$

Here, we use 8 hidden layers with 20 neural units and hyperbolic-tangent activation function.

```

In [3]: x = sn.Variable('x')
        t = sn.Variable('t')
        u = sn.Functional('u', [t, x], 8*[20], 'tanh')

```

PINNs no MATLAB

<https://www.mathworks.com/discovery/physics-informed-neural-networks.html>

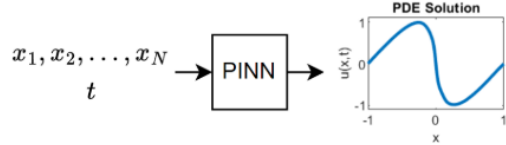
Documentação:

<https://www.mathworks.com/help/deeplearning/ug/solve-partial-differential-equations-with-lbfgs-method-and-deep-learning.html>

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} - \frac{0.01}{\pi} \frac{\partial^2 u}{\partial x^2} = 0,$$

with $u(x, 0) = -\sin(\pi x)$ as the initial condition, and $u(-1, t) = 0$ and $u(1, t) = 0$ as the boundary conditions.

This diagram illustrates the flow of data through the PINN.



Training this model does not require collecting data in advance. You can generate data using the definition of the PDE and the constraints.

Generate Training Data

Training the model requires a data set of collocation points that enforce the boundary conditions, enforce the initial conditions, and fulfill the Burger's equation. Select 25 equally spaced time points to enforce each of the boundary conditions $u(-1, t) = 0$ and $u(1, t) = 0$.

```
numBoundaryConditionPoints = [25 25];

x0BC1 = -1*ones(1,numBoundaryConditionPoints(1));
x0BC2 = ones(1,numBoundaryConditionPoints(2));

t0BC1 = linspace(0,1,numBoundaryConditionPoints(1));
t0BC2 = linspace(0,1,numBoundaryConditionPoints(2));

u0BC1 = zeros(1,numBoundaryConditionPoints(1));
u0BC2 = zeros(1,numBoundaryConditionPoints(2));
```

MathWorks® Products Solutions Learn Company

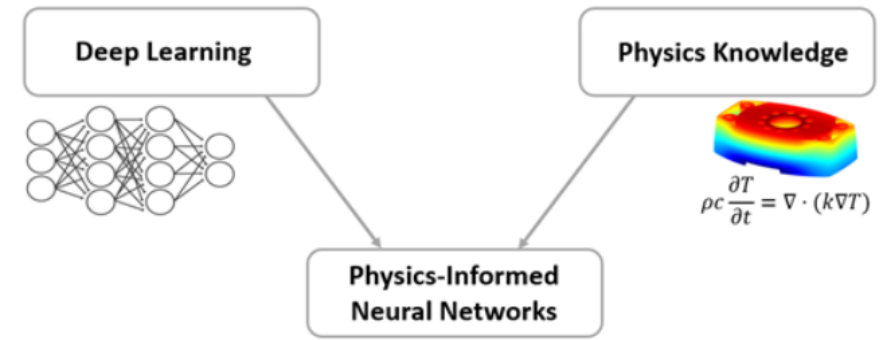
Physics-Informed Neural Networks (PINNs)

What Are Physics-Informed Neural Networks (PINNs)?

Physics-informed neural networks (PINNs) are **neural networks** that incorporate physical laws described by differential equations into their loss functions to guide the learning process toward solutions that are more consistent with the underlying physics. PINNs can be used to:

- Approximate solutions to **partial differential equations** (PDEs) and **ordinary differential equations** (ODEs).
- Solve **inverse problems**, such as estimating model parameters from limited data.

With Deep Learning Toolbox™, you can build and train PINNs, which enable rapid predictive analysis. You can integrate PINNs with MATLAB® and Simulink® for system-level simulation, control design, and design optimization.



Physics-informed neural networks (PINNs) include governing physical laws in the training of deep learning models to enable the prediction and modeling of complex phenomena while encouraging adherence to fundamental physical principles.



Aplicações em Mecânica Computacional

Frentes da Mecânica dos Sólidos

Problemas Diretos e Inversos



Frentes de Mecânica dos Sólidos

Grandes Áreas

Mecânica da Elasticidade Linear
 Mecânica da Elasto-plasticidade
 Mecânica da Hiper elasticidade
 Modelagem Viscoelástica
 Fratura Mecânica

Foco de Determinação

Deformações
 Tensões
 Deslocamentos
 Propriedades

Temas

Barras

Vigas

Euler-Bernoulli
 Timoshenko

Viga em Fundação Elástica

Placas

Placas de Kirchhoff-Love
 Funções de Airy
 Placa em Fundação Elástica
 Materiais elastoplasticos e hiperelásticos

Otimização Topológica

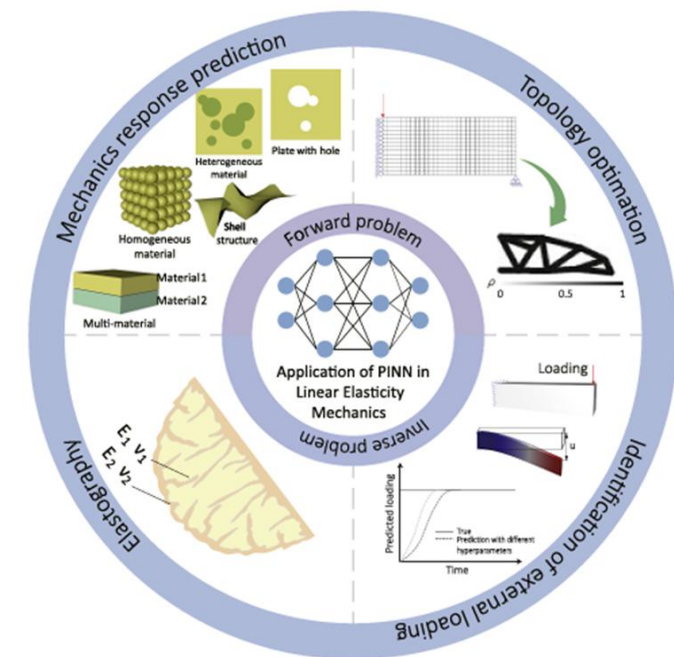
Problemas Dinâmicos

Vibrações
 Terremotos (prédios)

Transição de Campo de Fase

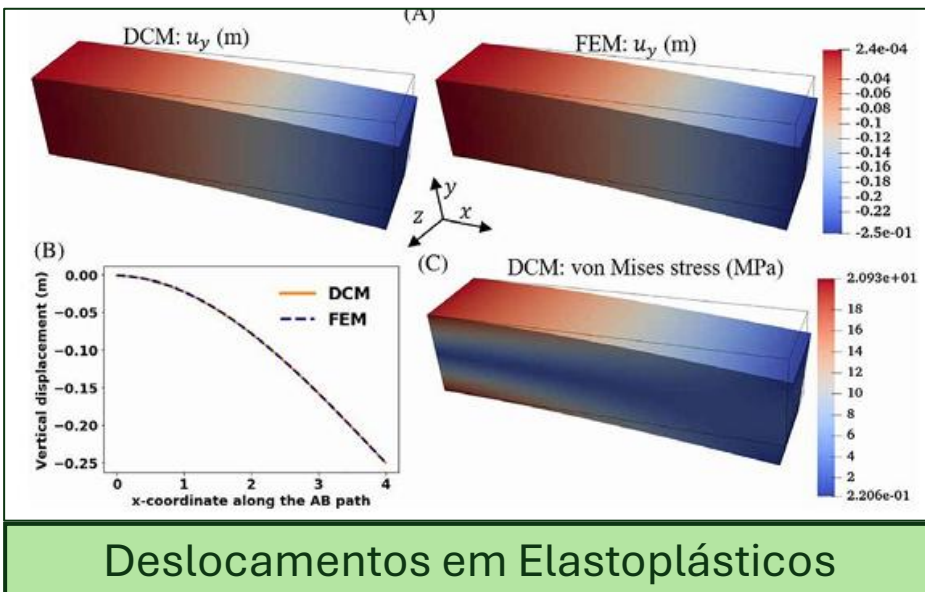
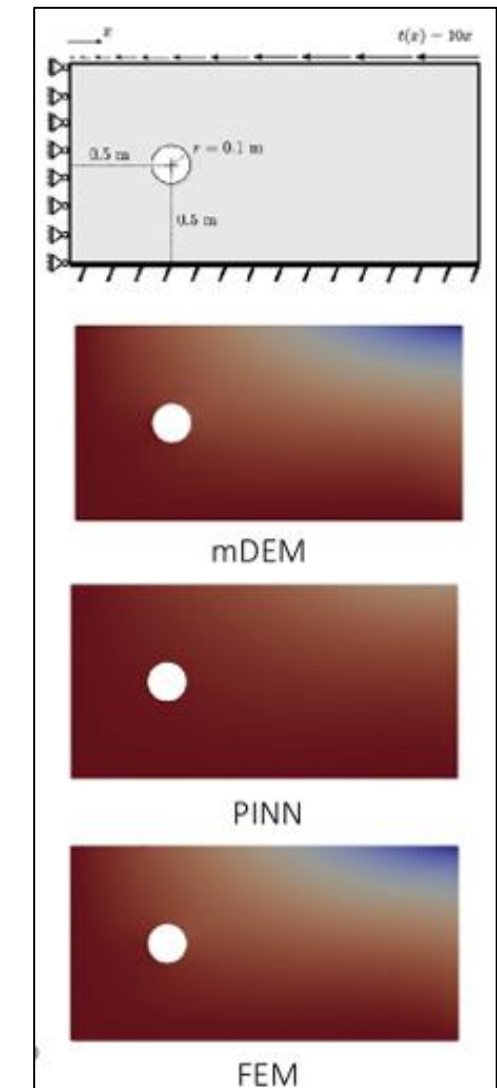
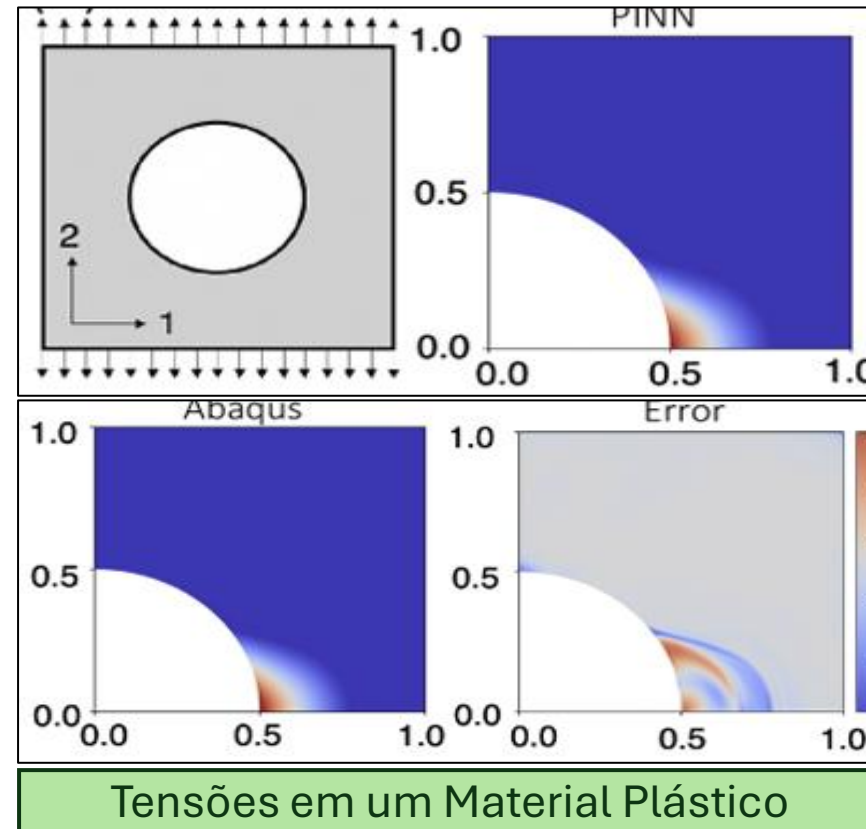
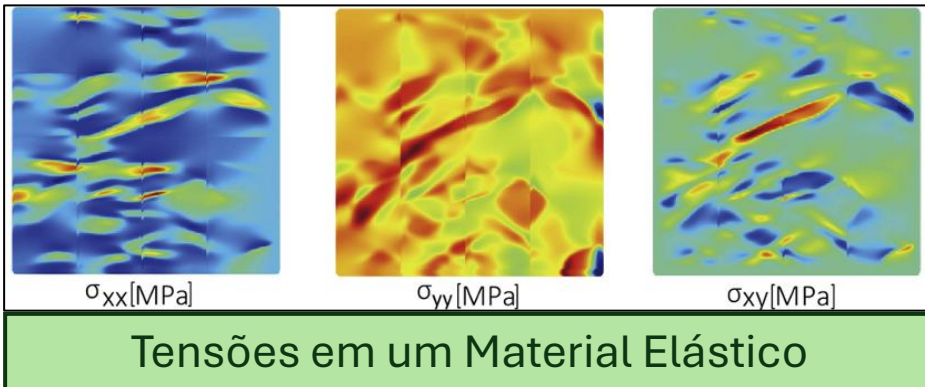
Identificações

Módulo de Elasticidade e Coeficiente de Poisson
 Equações Constitutivas
 Identificação de Defeitos

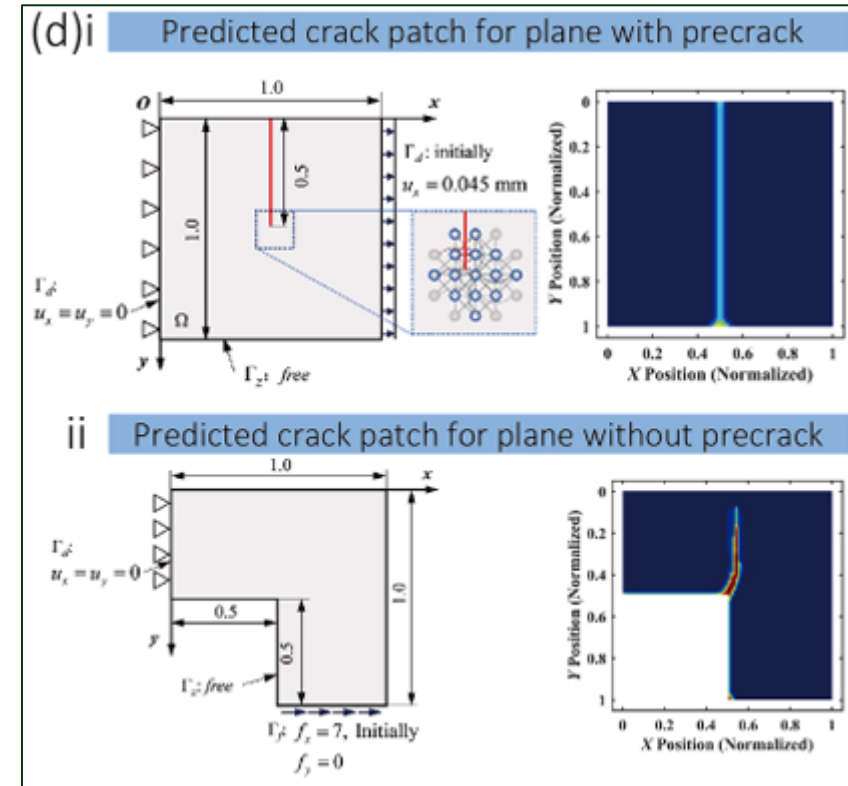
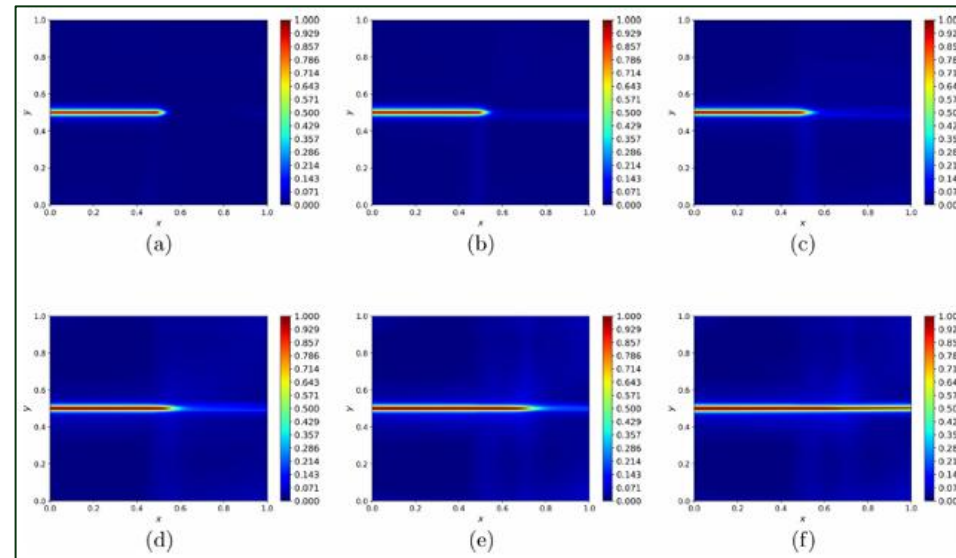
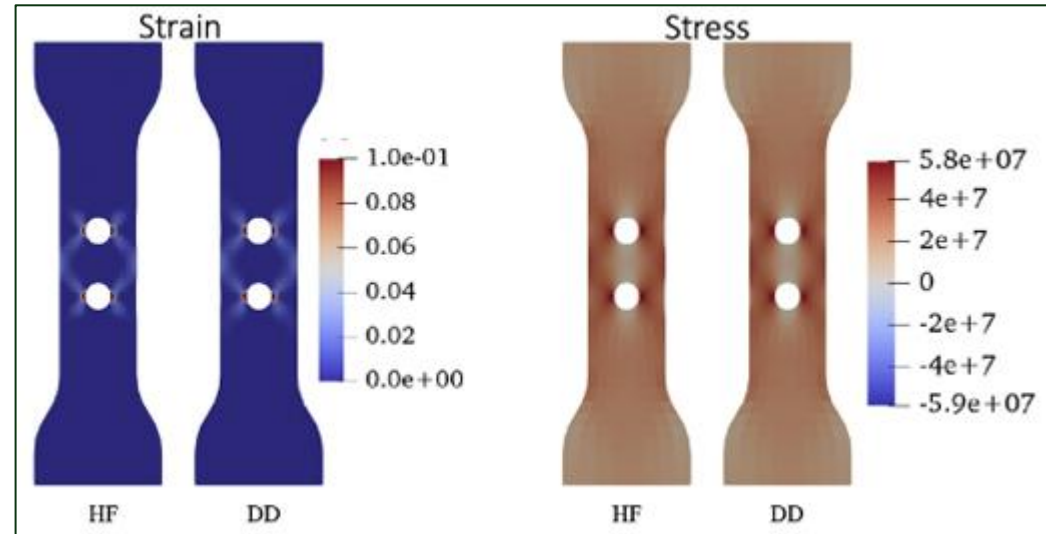


Hu et al (2024)

Problemas Diretos

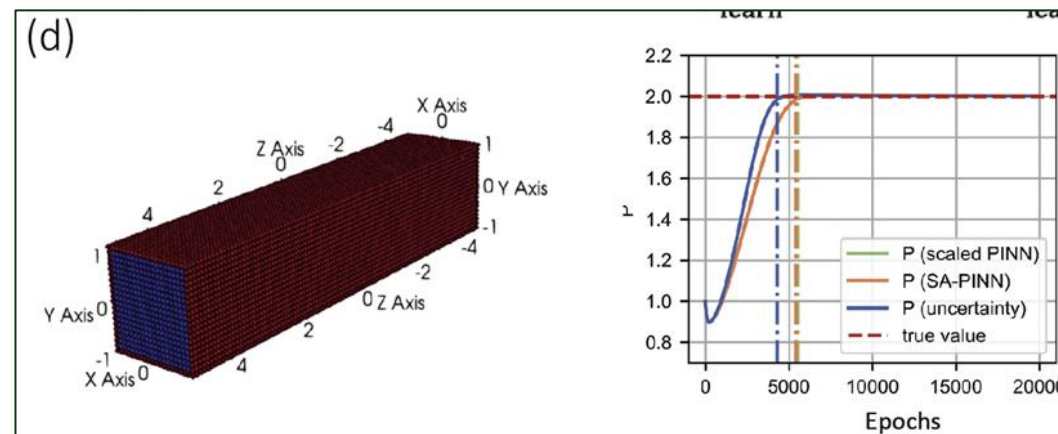
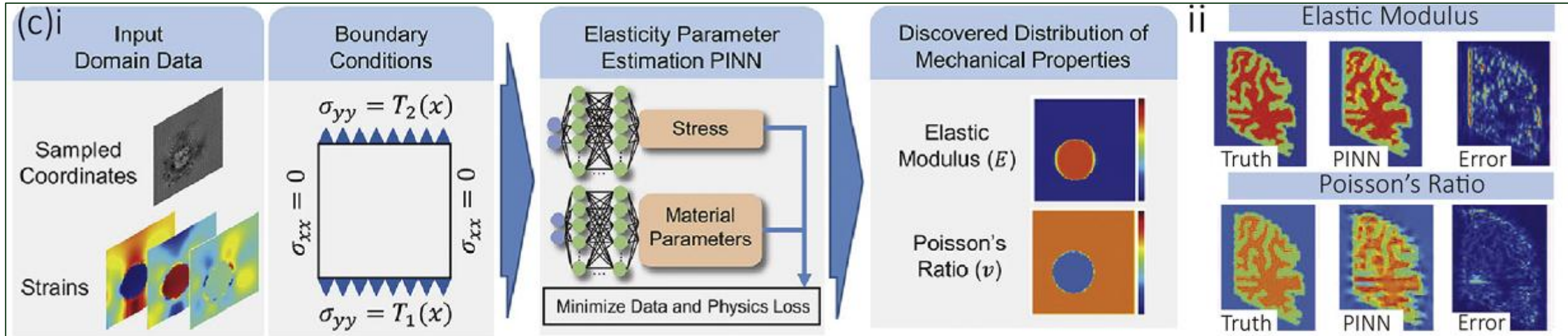


Problemas Diretos



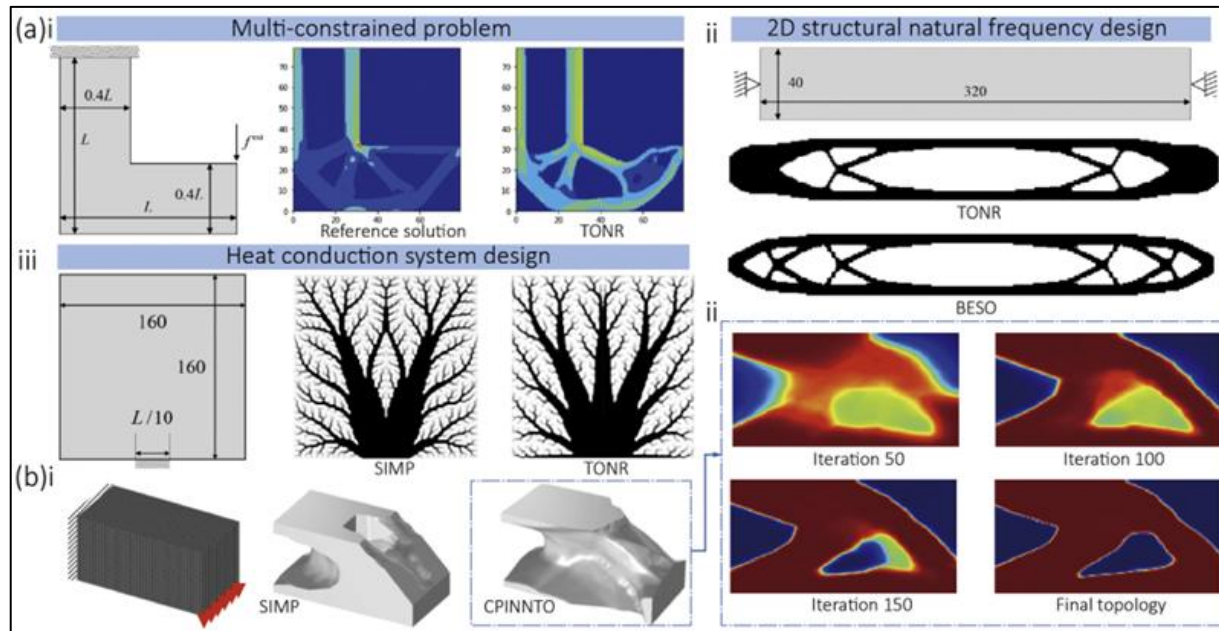
Propagação de Trincas e
Fratura Mecânica

Problemas Inversos



Identificação de Parâmetros

Problemas Inversos



Otimização Topológica

Case	(0) One elliptical void	(1) One elliptical void	(2) One slit-shaped void	(3) One circular void
Sketch (X_1 - X_2 plane)				
Unknown parameters θ	$X_1^{(c)}, X_2^{(c)}, A, B, \Gamma$	$X_1^{(c)}, X_2^{(c)}, A, B, \Gamma$	$X_1^{(1)}, X_2^{(1)}, X_1^{(2)}, X_2^{(2)}$	$X_1^{(c)}, X_2^{(c)}, R$

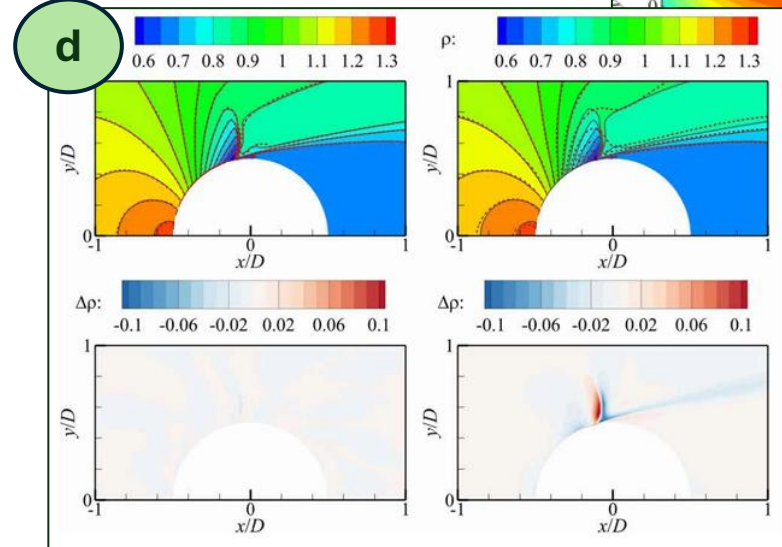
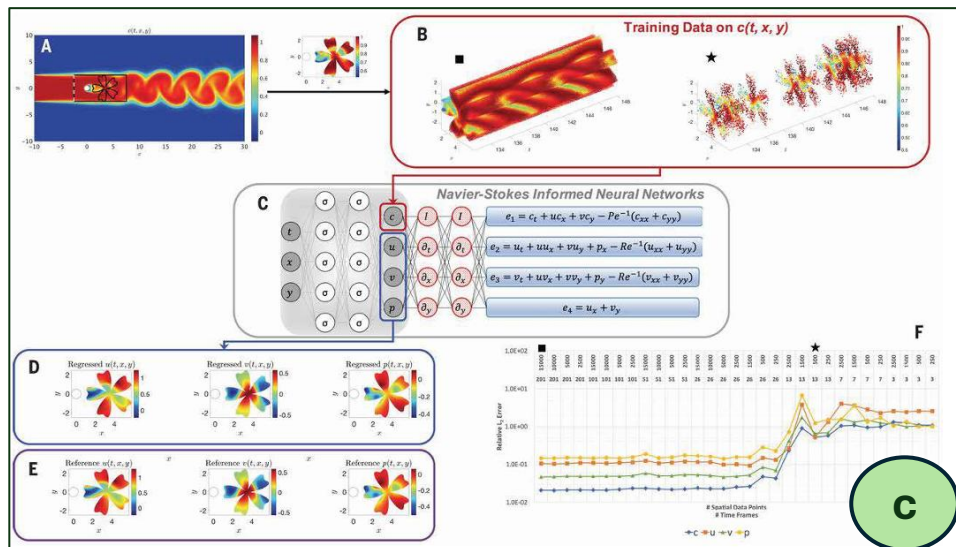
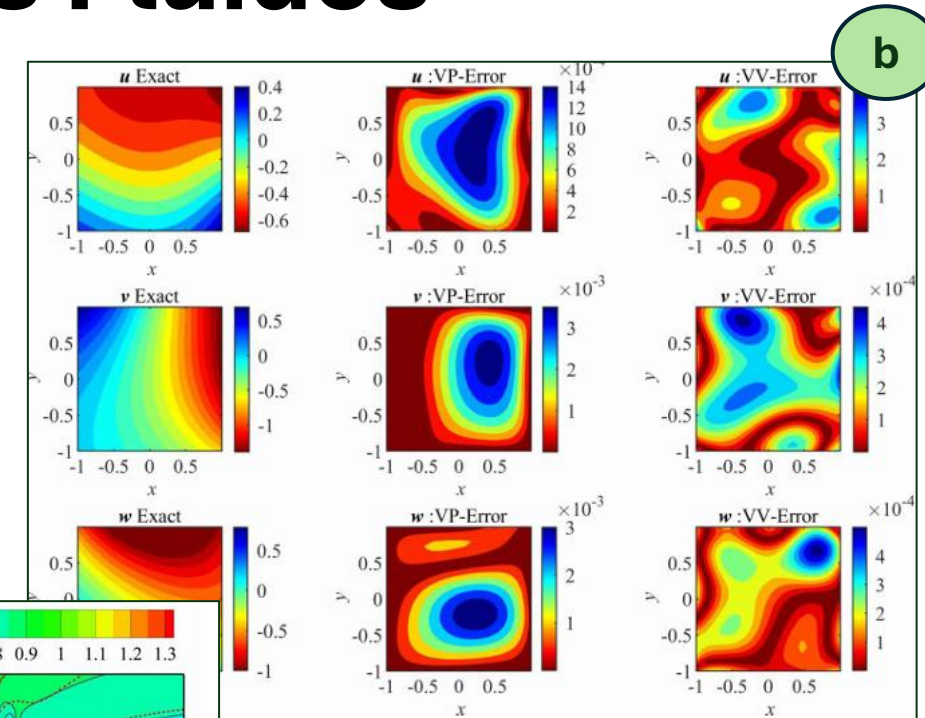
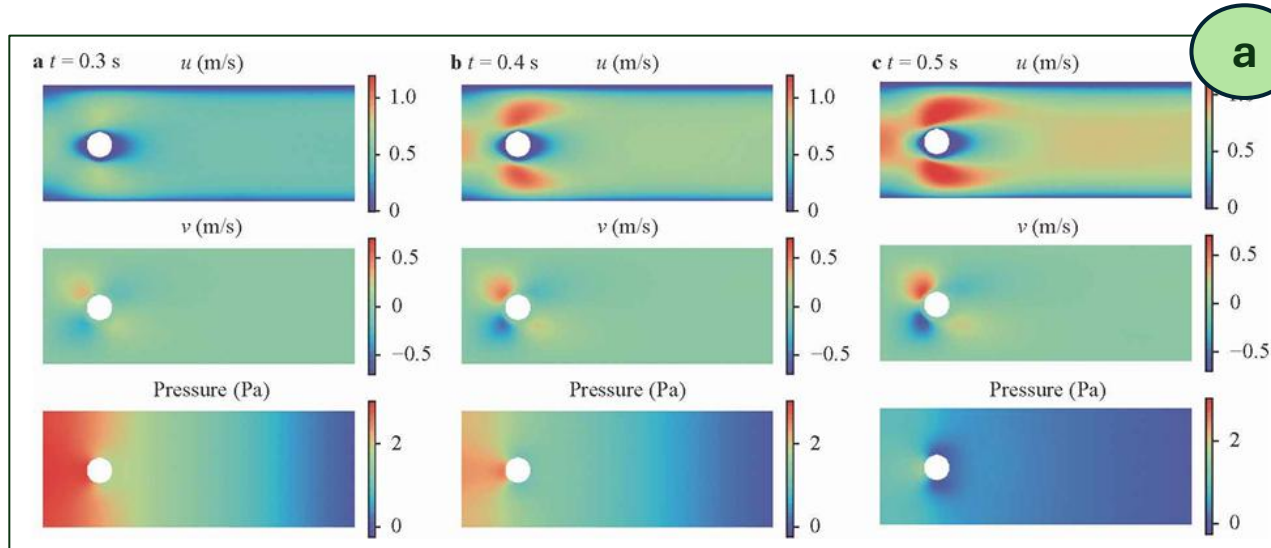
Case	(4) Two circular voids	(5) One circular inclusion
Sketch (X_1 - X_2 plane)		
Unknown parameters θ	$X_1^{(1)}, X_2^{(1)}, R^{(1)}, X_1^{(2)}, X_2^{(2)}, R^{(2)}$	$X_1^{(c)}, X_2^{(c)}, R, \mu_i$

Linear elasticity (compressible)	Matrix
Hyperelasticity (incompressible; Neo-Hookean)	Matrix
	Inclusion
Deformation plasticity (compressible; power law)	Matrix

Void

Identificação de Defeitos

Problemas em Mecânica dos Fluidos



- a) Fluido incompressível laminar
- b) Eq. Navier-Stokes
- c) Reconstrução de Campos
- d) Resolução de RANS



Vídeos e Referências

Vídeos Recomendados

Vídeos

Physics Informed Neural Networks (PINNs): "PyTorch" Solve Physical Systems with Deep Neural Networks: <https://youtu.be/MtHvs5FvtME?si=MeGVe9HF8pCkKN2j> <Acesso em: 13/05/2025>

Physics Informed Neural Networks for Fluid Mechanics:
https://youtu.be/EHrgSPHZG3Y?si=JUOWe1pC_7bt6i18 <Acesso em: 13/05/2025>

Playlists

Neural Networks / Deep Learning: <https://youtube.com/playlist?list=PLblh5JKOoLUIxGDQs4LFFD--41Vzf-ME1> <Acesso em: 03/06/2025>

Physics Informed Machine Learning:
<https://youtube.com/playlist?list=PLMrJAKhleNNQ0BaKuBKY43k4xMo6NSbBa> <Acesso em: 06/06/2025>

Physics Informed Neural Networks (PINNs):
<https://youtube.com/playlist?list=PLM7DTyYjZGuLmg3f6j40fEF18jyQmYsC2> <Acesso em: 06/06/2025>

Referências

Dwivedi (2019) Physics Informed Extreme Learning Machine (PIELM)–A rapid method for the numerical solution of partial differential equations

Estevão et al (2023) Physics-informed Neural Networks For Solving Elasticity Problems

Gao et al (2020): PhyGeoNet: Physics-Informed Geometry-Adaptive Convolutional Neural Networks for Solving Parameterized Steady-State PDEs on Irregular Domain

Gao et al (2022): Physics-informed graph neural Galerkin networks: A unified framework for solving PDE-governed forward and inverse problems

Haghighat et al (2020): A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics

Hu et al (2024): Physics-informed Neural Networks (PINN) for computational solid mechanics: Numerical frameworks and applications

Jagtap (2020): Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems

Referências

Jagtap (2021): Extended physics-informed neural networks (XPINNs): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations

Kharazmi (2019) VPINNs: Variational Physics-informed Neural Networks For Solving Partial Differential Equations

Kharazmi (2020) hp-VPINNs: Variational Physics-Informed Neural Networks With Domain Decomposition

Kollmannsberger et al (2021): Deep Learning in Computational Mechanics - An Introductory Course

Liu et al (2025) – KAN: Kolmogorov – Arnold Networks

Nentidis (2022) - Evaluation of Physics Informed Neural Networks in engineering simple structural analysis problems

Nguyen-Than (2019) - A deep energy method for finite deformation hyperelasticity

Referências

Raissi (2017) - Physics Informed Deep Learning (Part I): Data-driven Solutions of Nonlinear Partial Differential Equations

Ramabathiran (2021)- SPINN: Sparse, Physics-based, and partially Interpretable Neural Networks for PDEs

Samaniego (2019) - An Energy Approach to the Solution of Partial Differential Equations in Computational Mechanics via Machine Learning: Concepts, Implementation and Applications

Vu-Quoc & Humer (2023): Deep Learning Applied to Computational Mechanics: A comprehensive Review, State of the Art, and the Classics

Wang et al (2024a)- Artificial intelligence for partial differential equations in computational mechanics: A review

Wang et al (2024b)- Kolmogorov-Arnold-Informed neural network: A physics-informed deep learning framework for solving forward and inverse problems based on Kolmogorov–Arnold Networks



Obrigado!